

Organização e Arquitetura de Computadores

Anotações de aula

01 - Organização e Arquitetura de Computadores

Quando falamos sobre **arquitetura de computadores**, estamos nos referindo aos **atributos de um sistema que são visíveis a um programador** ou, explicando melhor, são atributos que têm um impacto direto sobre a execução da lógica de um programa. Por outro lado, quando falamos em **organização de computadores**, nos referimos às unidades operacionais e suas interconexões que percebem as especificações da arquitetura.

Em resumo, podemos entender a **arquitetura** e a **organização de computadores** através dos seguintes tópicos:

- **Arquitetura de computadores:**

- Atributos visíveis ao programador;
- Tem impacto direto na execução da lógica de um programa.

- **Atributos arquiteturais:**

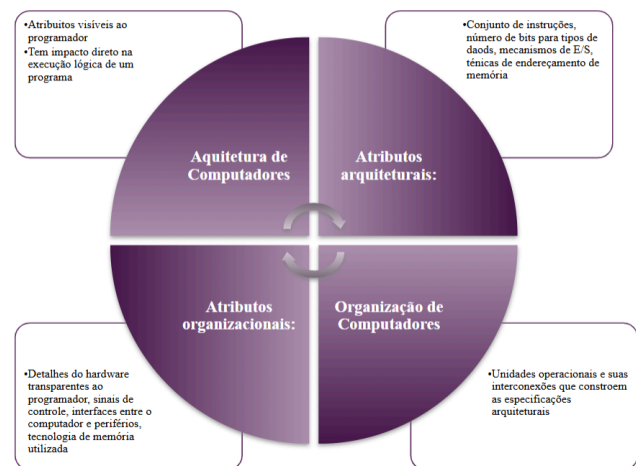
- Conjunto de instruções, números de bits para tipos de dados, mecanismos de E/S (entrada e saída), técnicas de endereçamento de memória, etc.

- **Organização de computadores:**

- Unidades operacionais e suas interconexões que constroem as especificações arquiteturais.

- **Atributos organizacionais:**

- Detalhes do hardware transparentes ao programador, sinais de controle, interfaces entre o computador e periféricos, tecnologia de memória utilizada.



Algo importante de se ter em mente é a analogia de que se um computador terá a instrução de multiplicação, isso diz respeito à arquitetura, mas se essa instrução será implementada por uma unidade especial de multiplicação ou pelo uso da unidade de adição repetidas vezes, isso diz respeito à organização.

Arquitetura da IBM System/370

Apresentada em 1970 e sobrevivendo até hoje como a arquitetura dos mainframes da IBM, ela é um ótimo exemplo de família arquitetural em que a variação entre modelos de computadores se dava apenas na organização, mas a arquitetura se mantinha a mesma.

Esse padrão incluía vários modelos de computadores que poderiam sofrer upgrades para modelos mais caros e mais rápidos sem precisar mudar a arquitetura e manter softwares feitos com uma organização anterior funcionando na nova configuração, evitando que o investimento em softwares não fosse perdido.

Estrutura e Função

Para podermos descrever e fazer o design de um computador, é essencial que reconheçamos sua estrutura altamente complexa e a natureza hierárquica dos sistemas mais complexos que, por sua vez, são constituídos por um conjunto de subsistemas inter-relacionados. Nesse contexto, o **projetista/designer só deve se preocupar com a estrutura e a função de um nível particular do sistema de cada vez**, para que possa fazer seu trabalho da melhor forma possível.

Definimos **estrutura** como sendo a maneira como os componentes se relacionam e **função** como a operação de componentes individuais como parte da estrutura, ou seja, o que cada componente que a compõe é responsável por fazer.

Função:

Em todo computador, existem 4 funções básicas que são essenciais para seu funcionamento adequado:

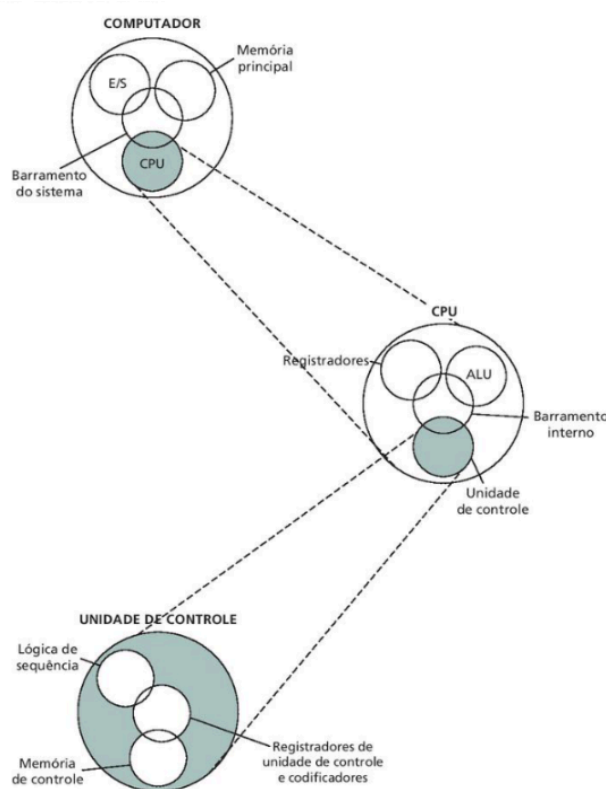
- **Processamento de dados:**
 - Dados podem ter uma variedade de formatos e a abrangência das necessidades de processamento é bem ampla.
- **Armazenamento de dados:**
 - Mesmo que o computador processe dados quase imediatamente, ainda é necessário armazenar pelo menos uma parte deles enquanto essa operação ocorre. Por isso, temos dois tipos de armazenamento: **curto prazo** e **longo prazo**.
- **Movimentação de dados:**
 - Entrada e Saída (E/S ou I/O): quando dados são recebidos de ou entregues a um dispositivo diretamente conectado ao computador (periféricos);
 - Comunicação de dados: quando dados são movidos entre longas distâncias, para ou de um dispositivo remoto.

- **Controle:**

- A unidade de controle **gerencia os recursos do computador e organiza o desempenho de suas partes** funcionais em resposta a **instruções**.

Estrutura básica:

Vamos analisar a estrutura básica de um computador tradicional com um processador único que emprega uma unidade de controle microprogramada.



- **CPU (Unidade Central de Processamento):** **Controla as operações de um computador e suas funções de processamento** de dados;

- **Memória principal:** Armazena dados;

- **E/S:** Movimenta os dados entre o computador e seu ambiente externo;

- **Interconexão:** mecanismo que **provê a comunicação entre a CPU, a memória principal e E/S**. Um exemplo é o barramento, que consiste em vários fios que interconectam as outras estruturas.

Por ser uma das estruturas mais importantes do computador, é de extrema importância que entendamos o funcionamento da **CPU, que é o “cérebro” da máquina**. Com base nisso, vejamos suas principais estruturas:

- **Unidade de controle:** **Controla todas as operações da CPU** e, por consequência, do computador;
- **Unidade Lógica-Aritmética (ULA):** Realiza o **processamento de dados** do computador;
- **Registradores:** Provém **armazenamento interno na CPU**;
- **Interconexões da CPU:** Mecanismos que provém **comunicação entre a unidade de controle, ULA e registradores**.

Estrutura multicore:

Chamamos de multicore os computadores que possuem chips com múltiplos processadores (chamados de core). Sua estrutura se dá da seguinte forma:

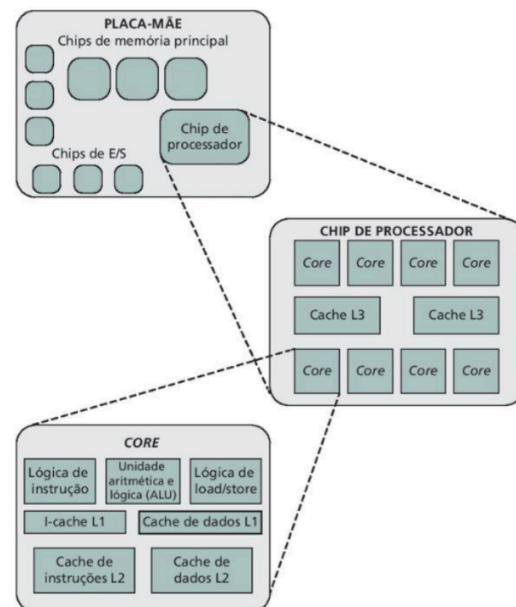
- **CPU:** É a parte que busca e executa instruções, consistindo em uma Unidade Lógica-Aritmética, uma unidade de controle e registradores. Ela só é chamada de processador em um sistema que possui apenas uma unidade de processamento;
- **Core:** Unidade de processamento individual em um processador, equivalente à funcionalidade da CPU em um sistema de uma CPU;
- **Processador:** Peça física de silício contendo múltiplas cores, sendo ele o componente do computador que interpreta e executa instruções.

Memória Cache:

Muito presente na grande maioria dos computadores contemporâneos (que fazem uso de múltiplas camadas de memória para que consigam otimizar ao máximo suas operações), a memória cache é uma unidade de memória que fica entre o processador e a unidade de memória principal, existindo com a finalidade de garantir um acesso mais rápido do processador a uma unidade de memória, já que é menor, mais rápida e está mais próxima fisicamente da estrutura que necessita dos dados armazenados nela (core).

Para otimizar ainda mais o acesso à memória, tornar as operações de um computador ainda mais rápidas e obter maior desempenho, uma estratégia é dividir memórias cache em múltiplos níveis, sendo o L1 o mais próximo do core e os L2, L..., Ln, mais distantes e, conseqüentemente, mais próximos da memória principal. É sempre importante frisar: **quanto mais próxima uma unidade de memória está do core, mais rapidamente o processamento que depende de seus dados é realizado.**

Visão simplificada dos principais elementos de um computador multicore.



História dos Computadores

Primeira geração: tubo de vácuo

Sendo usados para lógica digital (binária ou condicional), os primeiros computadores feitos faziam uso de válvulas a vácuo para possibilitar o funcionamento

dessa lógica. Como exemplo, temos o computador IAS, o computador mais famoso da primeira geração que estabeleceu o padrão de organização das estruturas que ainda é usado como modelo para a construção de máquinas nos dias atuais.

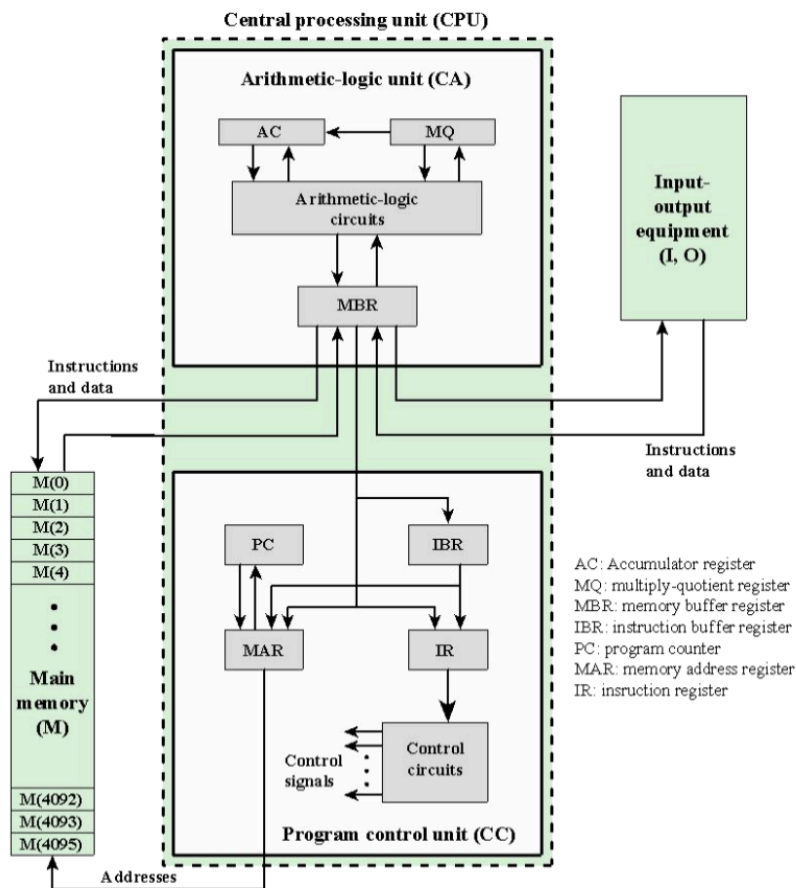


Figure 1.6 IAS Structure

Segunda geração: transistores

A segunda geração de computadores foi marcada pela substituição das válvulas a vácuo pelos transistores: uma alternativa desenvolvida nos Bell Labs em 1947 que era menor, mais barata, com uma dissipação de calor menor e que podia ser usada da mesma forma que a anterior. Além disso, ao invés de precisar de fios, metal, uma cápsula de vidro e do vácuo em si, os transistores eram dispositivos de estado sólido feitos de silício que dispensavam todos esses requisitos.

Essa nova geração de aparelhos trouxe consigo vários avanços e inovações para o mundo computacional, introduzindo:

- Unidades lógicas, aritméticas e de controle mais complexas que as anteriores;

- Uso de linguagens de programação de alto nível;
- Idealização de sistemas capazes de:
 - Carregar programas;
 - Mover dados para periféricos;
 - Bibliotecas para auxiliar desenvolvimento e computação.

Terceira geração: circuitos integrados

Como uma alternativa para facilitar a manufatura de computadores que faziam uso de transistores (unidades muito pequenas e complexas de serem colocadas manualmente nos circuitos), em 1958 surgem os circuitos integrados, cujos membros mais importantes dessa inovação foram o IBM System/360 e o DEC PDP-8.

Quando entendemos que as quatro operações principais de um computador são armazenamento, movimentação, processamento e controle de dados, podemos concluir que somente dois tipos de componentes são necessários para garantir o funcionamento adequado da máquina: **portas** (responsáveis por implementar uma função booleana ou lógica simples) e **células de memória** (que armazenam um bit de dados que representa seu estado: 1 ou 0). A partir disso, podemos relacionar nossas 4 operações principais da seguinte forma:

- Armazenamento: fornecido por células de memória;
- Processamento: fornecido pelas portas;
- Movimentação: os caminhos entre os componentes podem movimentar dados da memória para a memória e da memória pelas portas até a memória;
- Controle: o caminho entre os componentes pode movimentar sinais de controle.

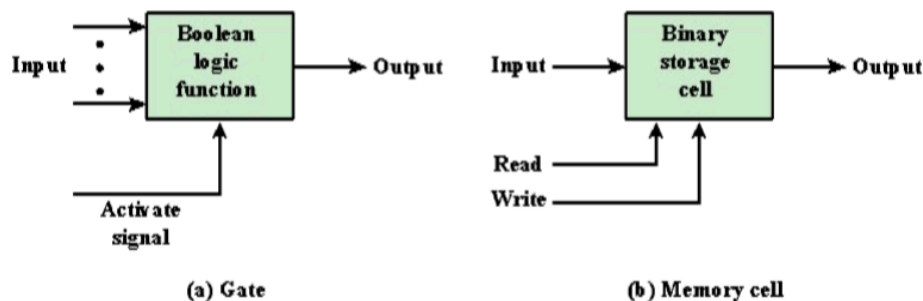


Figure 1.10 Fundamental Computer Elements

Considerando então que computadores são compostos de portas e célula de memória e esses componentes são feitos de componentes eletrônicos simples, surge a ideia de tirar vantagem do fato de que componentes como resistores, transistores e condutores podem ser feitos de semicondutores como o silício. Portanto, um circuito inteiro poderia ser fabricado em um pedaço de silício ao invés de montar componentes

individuais feitos do mesmo material no circuito. Isso faz com que muitos possam ser produzidos ao mesmo tempo em um único wafer de silício e conectados através de um processo de metalização para formar circuitos.

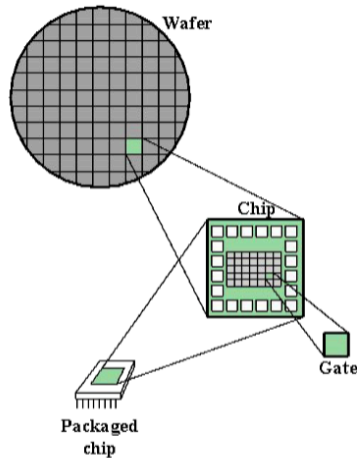


Figure 1.11 Relationship Among Wafer, Chip, and Gate

Na figura ao lado, temos a representação gráfica da ideia proposta anteriormente. Aqui temos um wafer fino de silício dividido em uma matriz de minúsculas áreas de poucos mm^2 . O padrão de circuito idêntico é fabricado em cada área e o wafer é dividido em chips. Cada chip consiste em muitas portas e/ou células de memória mais uma série de pontos de conexão de E/S. Esse chip então é encapsulado em um invólucro que o protege e oferece pinos de conexão para outros circuitos além do chip. Diversas dessas cápsulas podem ser então interconectadas em uma placa de circuito

impresso para produzir circuitos maiores e mais complexos sem precisar ser feita com componentes individuais manufaturados.

No começo, só algumas células e portas poderiam ser encapsuladas de modo confiável, mas com o passar do tempo, foi possível encapsular cada vez mais componentes em um mesmo chip. Percebendo esse crescimento, Gordon Moore propôs uma lei de que o número de transistores que poderia ser colocado em cada chip dobrava a cada ano, e esse ritmo se manteve até 1970, quando caiu para a cada 18 meses, mas sustenta essa faixa desde então.

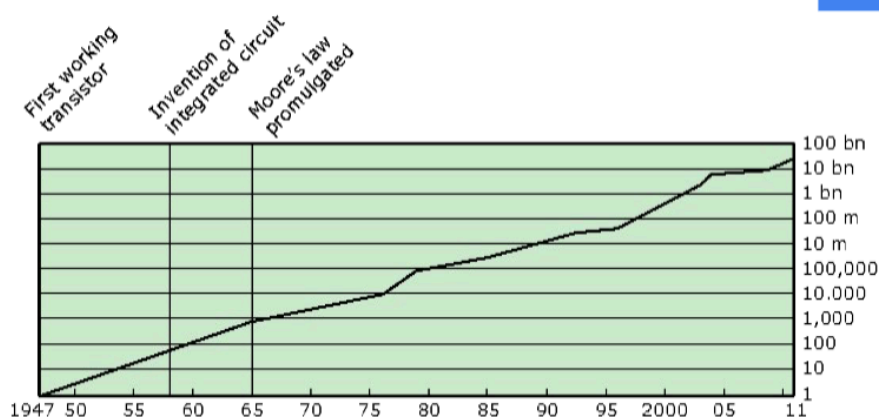


Figure 1.12 Growth in Transistor Count on Integrated Circuits (DRAM memory)

As consequências mais notáveis observadas pelo crescimento previsto na lei de Moore foram:

- Queda no custo de circuitos lógicos de memória em níveis extremos;
- Diminuição do caminho percorrido pelo sinal elétrico, aumentando a velocidade de operação;
- Diminuição no tamanho dos computadores, tornando mais conveniente seu uso em diversos ambientes;
- Redução na potência e requisitos para resfriamento;
- Menos conexões entre chips.

Próximas gerações

Após os avanços da terceira geração, foi possível desenvolver chips com integração em grande, muito grande e ultra grande escala, que permitiam a integração de, respectivamente, 1.000, 10.000 e mais de um bilhão de componentes.

Além disso, temos a chegada dos **microprocessadores**, cuja jornada se desenvolveu da seguinte forma:

- Após os avanços de integração, cada vez mais componentes eram integrados em chips, fazendo com que cada vez menos chips fossem necessários para construir um processador e, conseqüentemente, permitindo que eles tivessem seu tamanho reduzido;
- Em 1971 a Intel desenvolve o 4004, o primeiro chip que continha todos os componentes de uma CPU em um único artefato, sendo esse o nascimento do microprocessador;
- No ano seguinte, em 1972, a Intel lança o 8008, o primeiro microprocessador de 8-bits;
- É em 1974, então, que a Intel desenvolve o 8080, o primeiro microprocessador de propósito geral, sendo mais rápido, possuindo um maior conjunto de instruções e tendo uma capacidade ainda maior de endereçamento.

A partir daí, as evoluções não pararam, como podemos ver na tabela a seguir:

8080	8086	80286	80386	80486
• Primeiro microprocessador de propósito geral • 8-bit • Usado no primeiro computador pessoal (Altair)	• Máquina mais poderosa de 16-bits • Cache de instruções que faz prefetch • Surgimento da arquitetura x86 • 8088 foi a variante usada no primeiro computador pessoal da IBM	• Extensão do 8086 para endereçar 16-MB de memória (antes 1 MB)	• Primeiro processador 32-bit da Intel • Primeiro processador Intel multitarefas	• Introduz o uso de tecnologia de cache muito mais sofisticada e poderosa e pipeline de instruções • Coprocessador matemático embutido

02 - Desempenho

Design para desempenho

Conforme o custo de sistemas computacionais cai drasticamente com o passar do tempo, seu desempenho e a demanda por ele crescem no mesmo ritmo. Nesse contexto, os processadores se tornaram tão baratos que hoje são praticamente descartáveis, algo inimaginável para as gerações passadas.

Nesse contexto, empresas passam a se orientar na arquitetura cliente-servidor para atender uma vasta maioria das necessidades computacionais modernas e provedores de nuvem passam a usar bancos de servidores de alto desempenho para satisfazer o altíssimo volume de requisições em diversos espectros de aplicações e clientes.

Velocidade do Microprocessador

No contexto atual da alta demanda por desempenho e o boom dos microprocessadores, várias técnicas foram aplicadas para que a velocidade de processamento dos aparelhos fosse ainda maior. Entre elas, podemos citar:

- **Pipelining:** movimentação de dados e instruções em “pipes” (criação de canais unidirecionais temporários para a memória que permite o encadeamento de comandos no terminal);
- **Predição de desvios:** o processador prevê instruções à frente baseado em uma suposição da decisão de salto de um programa;
- **Execução superescalar:** capacidade de despachar mais de uma instrução por ciclo;
- **Análise de dataflow:** capacidade de avaliar quais instruções dependem de quais para criar um fluxo ótimo de execução sem dependências;
- **Execução especulativa:** usando predição de desvios e análise de dataflow, os processadores executam instruções de maneira especulativa antes mesmo de elas ocorrerem no fluxo do programa. Assim, mantém as unidades de execução o mais ocupadas possível.

Melhorias na Arquitetura e Organização de chips

Aumento na velocidade do chip:

- Consequência da redução do tamanho das portas lógicas:
 - Mais portas e portas mais densamente alocadas, culminando no aumento da velocidade do clock;

- Tempo de propagação de sinal reduzido.

Aumento do tamanho e velocidade das caches:

- Roubando parte do chip para a cache fez com que o tempo de acesso a ela ficasse reduzido e o tempo de processamento também.

Mudança na organização e arquitetura do processador:

- Aumento na velocidade efetiva da execução de uma instrução;
- Paralelismo.

Limitantes da velocidade de clock de um chip

Potência:

- A densidade de potência aumenta com o aumento da densidade do chip e velocidade de clock, fazendo com que haja maior dissipação de calor e, consequentemente, superaquecimento.

Delay RC:

- A velocidade com que os elétrons fluem é limitada pela resistência e capacitância dos caminhos no chip;
- A redução de tamanho nos componentes do chip causam redução do tamanho das conexões, aumentando a resistência;
- A redução da distância das conexões também aumenta a capacitância.

Latência de memória:

- A velocidade da memória está sempre abaixo da velocidade do processador.

Lei de Amdahl

Gene Amdahl propôs uma lei que lida com o potencial de **speed up** de um programa usando múltiplos processadores, em oposição a apenas 1. Ela ilustra os problemas com os quais a indústria lida para o desenvolvimento de arquiteturas multi-core: softwares devem ser adaptados para um ambiente de execução paralelo, assim aproveitando a estrutura multicore. Além disso, ela pode ser generalizada para avaliar ganhos de desempenho em softwares.

Sua fórmula consiste em: $t = (1 - f) * T + (T * f) / N$, sendo:

- **t**: tempo paralelizado;

- **f**: % do paralelismo (quanto pode ser otimizado);
- **T**: tempo levado sequencialmente;
- **N**: ganho paralelo (número de cores).

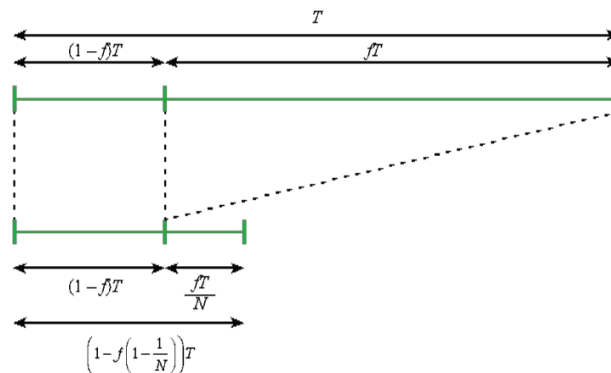


Figure 2.3 Illustration of Amdahl's Law

Vejamos um exemplo da aplicação da lei para calcular quanto tempo levará uma tarefa que leva 10s para ser concluída após a paralelização de 40% com 8 cores (ganho de desempenho):

- $t_{Par} = (1 - 0,4) * 10 + (0,4 * 10) / 8$
- $t_{Par} = 0,6 * 10 + 4 / 8$
- $t_{Par} = 6 + 1 / 2$
- $t_{Par} = 6,5s$

Nessas condições, podemos concluir que o tempo levado para a realização das tarefas após a paralelização é de 6,5s.

Agora, se quisermos calcular o **Speed Up** (ganho de velocidade -> quantas vezes um é mais rápido que o outro), usamos a seguinte fórmula: **T / tempo paralelizado**, em que T segue sendo o tempo original.

Latência e Throughput

Latência é **quanto tempo leva para fazer uma tarefa**, enquanto **Throughput** é a **quantidade de trabalho por unidade de tempo** (exemplo: tarefas por hora, transações por minuto, etc.). A seguir, veremos como latência e throughput são afetados pela troca de um processador por um mais rápido e pela adição de processadores, mas vamos focar na latência primeiro.

Desempenho relativo:

Desempenho = 1 / Tempo de execução, sendo geralmente interpretado como “X é n vezes mais rápido que Y”. Outra forma de medirmos o desempenho relativo é através da seguinte regra de três:

$$\frac{\text{Desempenho}_X}{\text{Desempenho}_Y} = \frac{\text{Tempo}_Y}{\text{Tempo}_X} = n$$

Por exemplo: se temos um programa P que leva 10s para executar um processo A e 15s para B, dizemos que o desempenho relativo de A sobre B é $\text{Tempo}_B / \text{Tempo}_A = 15s / 10s = 1,5$. Portanto, podemos concluir que A é 1,5 vezes mais rápido que B.

Medindo tempo de execução

Existem algumas formas de medir o tempo de execução de programas em computação. A primeira delas é usando o **tempo real** (relógio), que consiste no **tempo de resposta total**, incluindo todos os seus aspectos (como processamento, entrada e saída, tempo de espera, etc.). A segunda é usando o **tempo de CPU**, que consiste no **tempo gasto no processamento** (desconta E/S e outras tarefas paralelas), ou seja, **quantos ciclos foram usados por processo**, sendo dividido em tempo de usuário e tempo do sistema e considerando que programas diferentes são afetados de formas diferentes.

Clock da CPU

O clock consiste na taxa de atualização dos elementos de hardware da CPU, sendo que:

- **Período de clock:** consiste na **duração de um ciclo**;
 - Ex.: 250ps = 0.25ns = $250 \cdot 10^{-12}s$
- **Frequência de clock:** quantidade de **ciclos por segundo**.
 - Ex.: 4.0 GHz = 4000 MHz = $4 \cdot 10^9$ Hz

Tempo de CPU

O tempo de CPU, como dito, mede a quantidade de ciclos que foram usados em um processo. Portanto, podemos converter isso para a seguinte fórmula:

- **Tempo de CPU = #ciclos * período clock = #ciclos / frequência clock**, ou seja, $T = \#ciclos * T = \#ciclos / f$.

Quando levamos em consideração a melhoria de desempenho, seu resultado é obtido através da redução do número de ciclos e do aumento da frequência do clock, apesar de, normalmente, essas grandezas serem inversamente proporcionais.

Vejamos um exemplo de cálculo de tempo de CPU:

Suponhamos que temos um computador A cuja frequência de clock é de 2 GHz e o tempo de CPU é de 10s. Queremos então projetar um computador B que tenha 6s

de tempo de CPU e um clock mais rápido que eleve a quantidade de ciclos em 20%. Qual deverá ser o clock do computador B?

- $\text{tempo}_B = \text{ciclos}_B / \text{freq}_B \rightarrow \text{freq}_B = 1,2 * \# \text{ciclos}_A / 6s$
- $\text{tempo}_A = \# \text{ciclos}_A / \text{freq}_A \rightarrow \# \text{ciclos}_A = 10s * 2 \text{ GHz} = 20 * 10^9$
- $\text{freq}_B = 1,2 * 20 * 10^9 / 6s = 24 * 10^9 / 6 = \underline{4 \text{ GHz}}$

Contagem de instruções e CPI

A quantidade de ciclos de um processador (#ciclos) pode ser medida pela quantidade de instruções de um processo (#instruções) multiplicada pela quantidade de ciclos por instrução (CPI). Portanto, convertendo isso em fórmulas, temos: **#ciclos = #instruções * CPI**.

Se aplicarmos esse conceito à fórmula anterior de cálculo de tempo da CPU, teremos a fórmula completa: **Tempo CPU = #instruções * CPI * T = #instruções * CPI / f**.

A quantidade de instruções em um programa é determinada pelo próprio programa, ISA (arquitetura do conjunto de instruções), compilador e etc. Já a média de CPI é determinada pelo hardware da CPU. Por isso, se as instruções têm CPI diferentes, a média depende da composição de instruções.

Vamos ver um exemplo de cálculo de CPI:

Suponhamos que temos um computador A cujo período de clock é de 250ps com um CPI de 2,0, enquanto o computador B tem T = 500 ps e CPI de 1,2, sendo que ambos possuem a mesma ISA. Qual deles é mais rápido e quanto?

- $\text{tempo}_A = \# \text{instruções} * \text{CPI}_A * T_A = \#I * 2 * 250ps = \#I * 500ps$
- $\text{tempo}_B = \# \text{instruções} * \text{CPI}_B * T_B = \#I * 1,2 * 500ps = \#I * 600ps$
- A é mais rápido que B, já que leva menos tempo por instruções. Mas quantas vezes?
- $\text{tempo}_B / \text{tempo}_A = \#I * 600ps / \#I * 500ps = 1,2 = 20\% \text{ mais rápido.}$

CPI médio

Se instruções de **n** classes diferentes têm latências (em ciclos) diferentes, podemos definir que a quantidade de ciclos é a somatória do CPI * #instruções.

Para calcularmos o CPI médio, portanto, assumimos que a quantidade de ciclos por instrução é igual à quantidade de ciclos dividida pela quantidade de instruções, ou a somatória da CPI * (instrução / quantidade total de instrução).

Vejamos um exemplo de CPI e classes diferentes:

$$\# \text{ ciclos} = \sum_i (\text{CPI}_i \times \# \text{instruções}_i)$$

$$\text{CPI} = \frac{\# \text{ciclos}}{\# \text{instruções}} = \sum_i \left(\text{CPI}_i \times \frac{\# \text{instruções}_i}{\# \text{instruções}} \right)$$

Dados os programas 1 e 2 com classes de instruções A, B e C, qual é o CPI médio de cada um e qual dos dois programas é mais rápido?

Classe	A	B	C
CPI da classe	1	2	3
#instruções no programa 1	2	1	2
#instruções no programa 2	4	1	1

- Programa 1: IC = 5
- Programa 2: IC = 6
- $\#ciclos_{P1} = 2*1 + 1*2 + 2*3 = 10$
- $\#ciclos_{P2} = 4*1 + 1*2 + 1*3 = 9$
- CPI médio A = $10/5 = 2.0$
- CPI médio B = $9/6 = 1.5$
- continuar os cálculos...

03 - Estrutura do Computador

Componentes do Computador

Designs contemporâneos baseados nos conceitos apresentados por John von Neumann:

Arquitetura von Neumann:

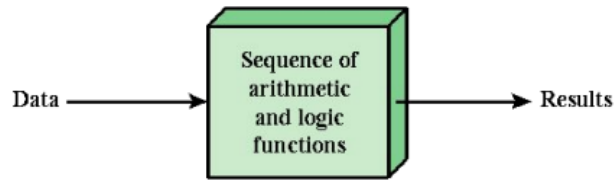
- **Dados e instruções** armazenados em um **memória única**;
- O **conteúdo** dessa memória é **endereçável por localização**, sem se preocupar com o tipo de dado existente;
- A execução ocorre de maneira sequencial (de uma instrução para a próxima).

Programa hardwired:

- Resultado da interconexão entre vários componentes de hardware na configuração desejada para a execução de instruções;
- É considerado o predecessor do software.

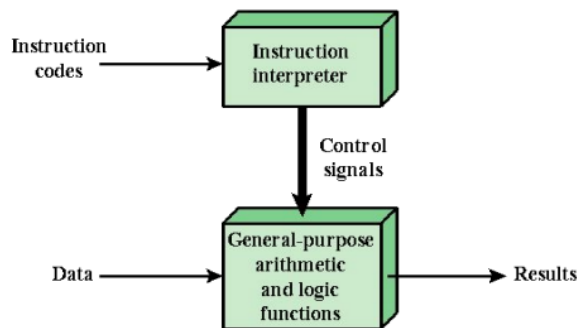
Abordagem de Hardware e Software

Quando programamos em hardware, temos um fluxo de execução que consiste na **entrada de dados** e no **processamento** desses dados através de uma **sequência de funções lógicas aritméticas**, que então expõe como **saída** os **resultados** desejados oriundo do processamento dos dados recebidos.



(a) Programming in hardware

No entanto, quando começamos a trabalhar com software, esse fluxo sofre algumas alterações: surgem agora os **códigos de instruções**, que são **convertidos pelo interpretador de instruções em sinais de controle**. Esses sinais atuam como entrada no bloco que novamente recebe dados, mas agora também **recebe os sinais para processar ambos através de funções lógicas e aritméticas de propósito geral** (como soma, multiplicação, alocação de memória, etc), para então devolver os resultados.



(b) Programming in software

Software:

- Sequência de **código** ou **instruções**;
- Parte do hardware interpreta as instruções e gera sinais de controle;
- Inserimos uma nova sequência de códigos para cada programa no lugar de construir o novo hardware.

Componentes principais:

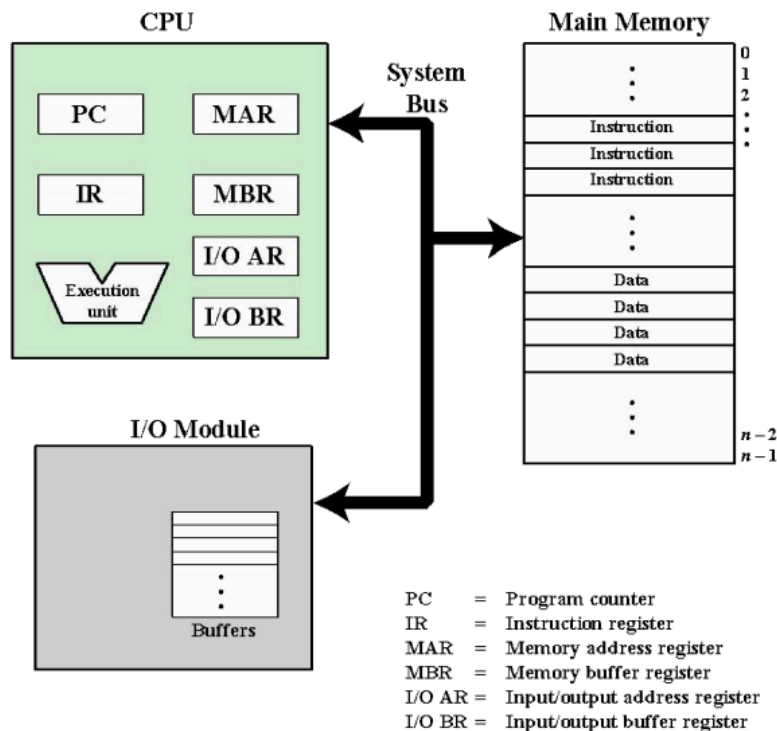
- **CPU:**
 - **Interpreta instruções**;
 - Apresenta **funções lógicas e aritméticas de propósito geral**.
- **Componentes de E/S:**
 - Módulo de entrada:
 - Contém componentes básicos para **receber dados e instruções e convertê-los em sinais internos** utilizáveis pelo sistema.
 - Módulo de saída:

- Mecanismo para reportar os dados.

- **Memória/Registradores:**

- Registradores são **espaços de memória dentro da CPU**, o que faz com que **operações sejam mais rápidas** de serem executadas devido à distância menor que precisa ser percorrida. Num computador, os 4 tipos principais de registradores que temos são:
- **Registrador de endereço de memória (MAR):**
 - Endereço em memória para a **próxima escrita/leitura** de dados.
- **Registrador de buffer de memória (MBR):**
 - Contêm **dados a serem escritos** ou espaços que **armazenam os dados lidos** em memória.
- **Registrador de endereço de E/S (I/OAR):**
 - **Aponta** para um **dispositivo de E/S** específico.
- **Registrador de buffer de E/S (I/OBR):**
 - Usado para **troca de dados** entre módulos de **E/S e a CPU**.

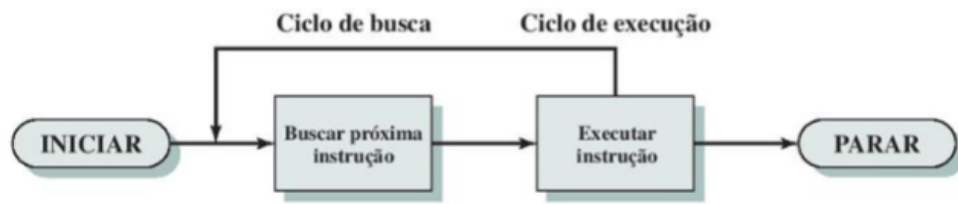
Visão de alto-nível dos componentes de um computador:



O contador de programa (PC) busca as instruções da memória e as armazena no registrador de instruções (IR), incrementando seu valor a cada nova instrução buscada;

Ciclo de fetch/busca:

1. No início de cada ciclo de instrução, o **processador busca uma instrução** na memória;
2. Em seguida, **o contador de programa (PC) armazena o endereço da instrução** que devemos **buscar depois**;
3. O processador incrementa o PC após cada busca de instrução, para garantir que a próxima busca seja a instrução na sequência;
4. A instrução buscada é então carregada no registrador de instrução (IR);
5. O processador interpreta a instrução e computa a ação requisitada.



Categorias de ações realizadas pelo processador:

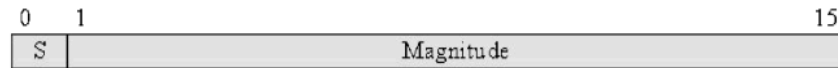
- **Processador-memória:**
 - Dados transferidos entre o processador e a memória.
- **Processador-E/S:**
 - Dados transferidos entre o processador e os dispositivos periféricos.
- **Processamento de dados:**
 - Processamento **lógico/aritmético** dentro do processador.
- **Controle:**
 - Instruções que especificam a **alteração no fluxo de execução**, **mudando** o comportamento padrão do **PC** de sempre ir para a instrução seguinte (ocorre quando temos estruturas condicionais, loops, etc. → tudo o que não for sequencial).

A execução de uma instrução pode envolver uma combinação dessas ações.

Vamos considerar um exemplo simples usando uma máquina hipotética que inclui as características da figura a seguir. Um processador contém um único registrador de dados, chamado acumulador (AC). **Instruções e dados possuem 16 bits de extensão**. Assim, é conveniente **organizar a memória** usando palavras de 16 bits. O formato de **instruções oferece 4 bits para o Opcode** (Opcode é o nome dado ao código/identificador da instrução, sendo responsável por indicar para a CPU se aquela instrução é uma soma, multiplicação, etc.), de modo que pode haver até $2^4 = 16$ opcodes diferentes, e até $2^{16} = 4096$ (4K) palavras de memória (address/endereço de memória) podem ser endereçadas diretamente.



(a) Instruction format



(b) Integer format

Program Counter (PC) = Address of instruction
 Instruction Register (IR) = Instruction being executed
 Accumulator (AC) = Temporary storage

(c) Internal CPU registers

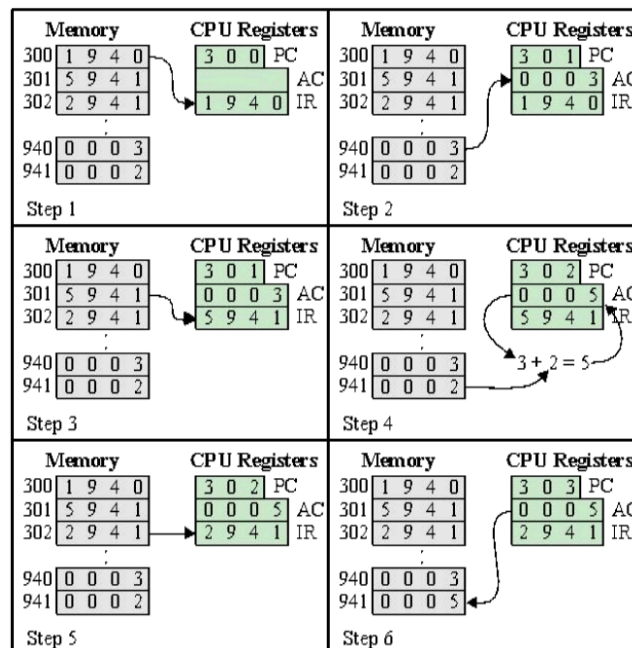
0001 = Load AC from Memory
 0010 = Store AC to Memory
 0101 = Add to AC from Memory

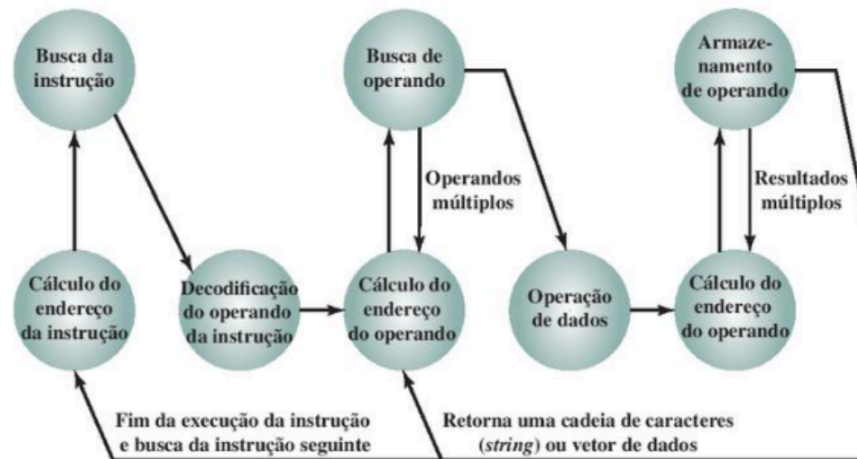
(d) Partial list of opcodes

Figure 3.4 Characteristics of a Hypothetical Machine

S (sinal de inteiro) -> 1 bit é reservado para armazenar o sinal do número inteiro;
 Magnitude -> 15 bits são reservados para armazenar o número inteiro em si.

Vejamos abaixo um exemplo da execução de um programa e o diagrama de estado de ciclo de uma instrução:





Funções de Entrada e Saída (E/S):

Módulos de entrada e saída podem trocar informações diretamente com o processador.

O processador pode ler ou escrever dados em um módulo de E/S, de forma que ele identifica um dispositivo específico controlado por um módulo de E/S particular e referencia instruções de E/S no lugar de instruções de memória. Ademais, **módulos de E/S podem trocar dados diretamente com a memória, visando reduzir o tempo de CPU da transmissão de dados**. É permitido ao dispositivo se conectar com a memória por intermédio do processador, e essa prática é chamada de **DMA (direct memory access)**.

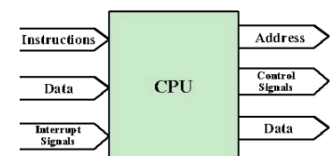
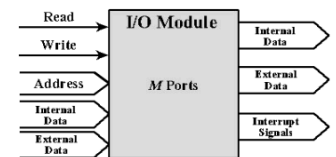
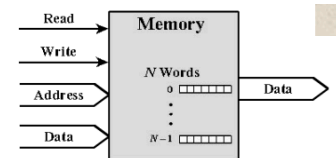


Figure 3.15 Computer Modules

Estruturas de conexão:

Estruturas de conexão devem lidar com os seguintes tipos de transferência de dados:

- **Memória para o processador:**
 - O processador lê as instruções ou dados da memória.
- **Processador para a memória:**
 - O processador escreve uma unidade de dados na memória.
- **E/S para o processador:**
 - O processador lê dados de um dispositivo de E/S do módulo de E/S.
- **Processador para E/S:**
 - O processador envia dados para um dispositivo de E/S.

- **E/S para e da memória:**
 - Um módulo de E/S troca dados diretamente com a memória.

Conexões de barramento:

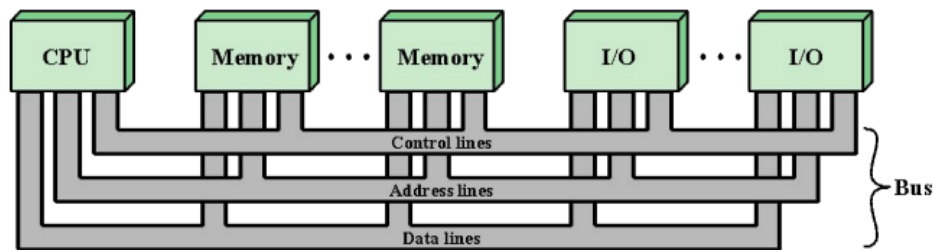
- **Caminho de conexão entre dispositivos;**
 - Barramento de transmissão compartilhado.
- Sinais transmitidos por um dispositivo ficam disponíveis para todos os dispositivos;
 - Se dois dispositivos transmitem ao mesmo tempo, os sinais se quebram.
- Consiste tipicamente de várias linhas de comunicação;
 - Cada linha transmite sinais representando 1 e 0.
- Sistemas computacionais contém um número de barramentos diferentes que **provêm caminhos entre componentes de diversos níveis da hierarquia do computador;**
- **Barramento do sistema;**
 - **Conecta os principais componentes do sistema** (CPU, memória, E/S).
- A grande maioria das estruturas de interconexão é baseada em um ou mais barramentos.

Barramentos:

- **Barramento de Dados:**
 - Barramentos são linhas de dados que **provêm caminhos para a transmissão de dados entre os componentes de um computador.** Consiste de 32, 64, 128 ou mais linhas, sendo que o número de linhas equivale à largura do barramento e determina quantos bits podem ser transferidos ao mesmo tempo. A largura, portanto, é essencial para nivelar o desempenho do sistema.
- **Barramento de Endereçamento:**
 - Designa a **fonte e destino dos dados no barramento de dados.** Se o processador deseja ter uma palavra de dados da memória, ele coloca os endereços da palavra nas linhas de endereçamento;
 - **A largura determina o tamanho máximo possível da capacidade de memória do sistema** e é usada para endereçar as portas de E/S, uma vez que bits mais significativos selecionam um módulo particular no barramento e os menos significativos selecionam a localização do elemento dentro do módulo.

- **Barramento de Controle:**

- Usado para controlar o acesso do barramento. Como os barramentos de dados e endereçamento são compartilhados, precisamos sincronizar seu uso;
- Sinais de controle transmitem comandos e informações de temporização entre os módulos conectados;
- Sinais de temporização indicam o prazo de validade das informações de dados e endereços;
- Sinais de comando especificam operações a serem executadas.



Interconexão Ponto-a-Ponto:

A interconexão ponto-a-ponto surgiu dos limites elétricos encontrados em frequências mais altas de transmissão de dados. Em níveis mais altos de transmissão de dados, fica cada vez mais difícil elaborar a sincronização de todos os módulos. Portanto, um único barramento convencional no mesmo chip destaca as dificuldades conforme maiores transmissões de dados eram exigidas para acompanhar o crescimento do desempenho dos processadores. Nesse contexto, a interconexão ponto-a-ponto possui menor latência, maior transmissão de dados e melhor escalabilidade.

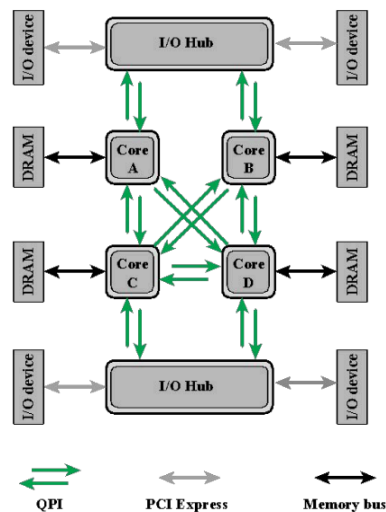


Figure 3.17 Multicore Configuration Using QPI

04 - Introdução ao Assembly MIPS

Assembly → linguagem de programação de baixo nível que trabalha com mnemônicos em linguagem natural;

Assembler → software que converte o código escrito em assembly para código binário.

Características de instruções

A operação do processador é determinada pelas instruções que ele executa, chamadas de instruções de máquina ou instruções de computador (ou, aqui, somente instruções). A coleção de instruções que um processador pode executar é o conjunto de instruções (ISA: Instruction Set Architecture). Cada instrução carrega uma quantidade de informação requerida pelo processador para executar alguma coisa.

Elementos de uma instrução:

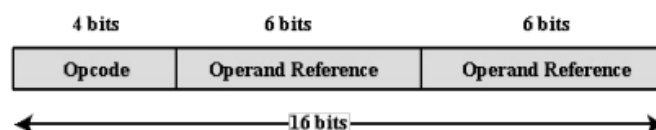
- Código da operação (opcode);
- Referência para operando(s) de entrada;
- Referência para operando(s) de saída;
- Referência para a próxima instrução.

Possíveis locais de operandos:

- **Memória:** a instrução contém o endereço de memória para o operando;
- **Dispositivo de E/S:** a instrução contém uma codificação identificando o módulo de E/S utilizado. É comum que E/S seja mapeada como uma região de memória;
- **Registrador interno:** o processador possui registradores que podem ser usados como operandos. Se houver mais do que um registrador, a instrução contém um código que identifica qual é usado como operando;
- **Imediato:** o valor do operando está codificado diretamente na instrução.

Representação de uma instrução

No computador, uma instrução é representada por uma sequência de bits. Ela é dividida em campos, correspondendo aos elementos que a constituem.



Opcodes são representados por abreviações chamadas de **mnemônicos**. Ex.:

- ADD (Add) - Somar;
- SUB (Subtract) - Subtrair;
- MUL (Multiply) - Multiplicar;
- DIV (Divide) - Dividir;
- LOAD (Load data from memory) - Carregar da memória;
- STORE (Store data to memory) - Salvar na memória.

Operandos também são representados por **símbolos**, de forma que cada um possui uma representação binária fixa. Ex.:

- ADD R, Y ou add \$t0, \$t1, \$t2

Tipos de Instrução:

- **Processamento de dados:**
 - Operações aritméticas;
 - Operações lógicas.
 - (Ex.: soma, subtração, etc)
- **Armazenamento de dados:**
 - Movimento entre registradores e memória.
 - (Ex.: carregamento e armazenamento na memória)
- **Movimentação de dados:**
 - Operações de E/S.
- **Controle:**
 - Operações de teste;
 - Operações de desvio.
 - (Ex.: loops e condicionais)

Arquitetura de instruções RISC X CISC:

- **RISC (Reduced Instruction Set Computer):**
 - Instruções simples;
 - Construído para ser simples e rápido;
 - Referências só com LOAD/STORE;
 - Instruções de formato fixo;
 - Poucas instruções com poucos formatos de endereçamento;
 - Compilador complexo;
 - Vários registradores.
- **CISC (Complex Instruction Set Computer):**
 - Instruções complexas;
 - Construído para se parecer com linguagens de alto nível;
 - Qualquer instrução pode referenciar à memória;

- Instruções com formato variável;
- Muitas instruções com muitos modos de endereçamento;
- A complexidade está no programa;
- Poucos registradores.

O Conjunto de Instruções MIPS

Tipos de operandos:

- Endereços;
- Números;
- Dados lógicos;
- Caracteres.

Data Transfer:

Data Transfer é o tipo mais fundamental de instrução, sendo responsável por identificar a localização dos operandos, o tamanho dos dados e o modo de endereçamento de cada operando.

Aritmética:

A maioria das máquinas oferecem operações aritméticas básicas (+, -, *, /). As operações básicas trabalham com números inteiros com sinal, mas podem também utilizar ponto flutuante, BCD, etc. Outras operações incluem valor absoluto, negação, incremento e decremento.

Operações aritméticas

Todas as operações aritméticas utilizam **3 operandos**: 2 entradas e uma saída. O primeiro operando é sempre o que recebe o resultado da operação, enquanto os outros dois serão usados na operação.

- **Soma:**
 - **add** a, b, c # $a \leftarrow b + c$
- **Subtração:**
 - **sub** a, b, c # $a \leftarrow b - c$

Vejamos um exemplo da conversão da expressão matemática em C: **$f = (g + h) - (i + j)$** para MIPS:

1. **add** t,g,h # temp t = g + h
2. **add** t2,i,j # temp t2 = i + j
3. **sub** f,t,t2 # temp f = t - t2

Operandos em registradores

Instruções aritméticas usam operandos em registradores. O MIPS tem 32 registradores de 32 bits cada. Suas principais características incluem:

- São utilizados para dados frequentemente acessados;
- Numerados de 0 a 31;
- Dados de 32 bits são uma “palavra”;
- R0 sempre é zero (\$zero).

Nomes no Assembler:

- **\$t0, \$t1, ..., \$t9**: são usados para **valores temporários**;
- **\$s0, \$s1, ..., \$s7**: são usados para **variáveis salvas**.

Agora vamos ver como ficaria conversão da mesma expressão para MIPS fazendo uso de registradores, considerando a tabela ao lado:

1. **add \$t0,\$s1,\$s2** # t0 = g +h
2. **add \$t1,\$s3,\$s4** # t1 = i+j
3. **sub \$s0,\$t0,\$t1** # f = t0 + t1

f	\$s0
g	\$s1
h	\$s2
i	\$s3
j	\$s4

Operandos em memória:

A memória principal é usada para dados estruturados, como vetores, estruturas de dados, alocação dinâmica, etc. Para fazer operações aritméticas, usaremos as funcionalidades:

- Carregar (load) os valores da memória em registradores;
- Salvar (store) os resultados de volta na memória.

A memória é endereçada por bytes e cada endereço de memória aponta para um dado de 8 bits.

Como a memória sempre é acessada de 4 em 4 bytes, palavras alinhadas na memória possuem endereços exclusivamente em múltiplos de 4.

lw (load word -> carregar em registradores)

sw (store word -> armazenar em memória)

```
lw  $s1, 20($s2)  # $s1 ← MEMÓRIA($s2 + 20)
sw  $s1, 20($s2)  # MEMÓRIA($s2 + 20) ← $s1
```



Vejamos o seguinte exemplo com a conversão da expressão matemática em C: **g = h A[8]** para MIPS, usando como base a tabela de registradores dada:

g	\$s1
h	\$s2
&A	\$s3

1. **lw** \$t0, 32(\$s3) # salva no registrador o valor da posição 8 do vetor A, que está 8*4 bytes deslocado a partir do início do vetor, apontado por \$s3 ($t0 = A[8]$)

2. **add** \$s1, \$s2, \$t0 # $g = h + t0$

Vamos conferir mais um exemplo com a conversão a expressão matemática em C: **A[12] = h A[8]** para MIPS, usando como base a tabela de registradores:

h	\$s2
&A	\$s3

1. **lw** \$t0, 32(\$s3) # $t0 = A[8]$

2. **add** \$t0, \$t0, \$s2 # $t0 = t0 + h$

3. **sw** \$t0, 48(\$s3) # Guarda na memória o novo valor calculado na posição que é $12*4$ bytes a partir do início do vetor

Registradores vs Memória:

Registradores são mais rápidos de acessar, enquanto a memória requer loads e stores e, consequentemente, mais instruções para serem executadas.

O compilador deve usar registradores o máximo possível e só recorrer à memória para variáveis menos utilizadas. No entanto, otimizar o uso de registradores é importante, ainda mais levando em conta que temos uma quantidade limitada deles.

Operandos em imediatos

Imediatos são constantes embutidas na instrução, conforme mostra o exemplo a seguir: **addi** \$s3, \$s3, 4

Não existe subtração com imediatos, portanto, para realizar esse tipo de operação, usamos o valor do imediato com o sinal negativo. Ex.: **addi** \$s3, \$s3, -1

Usar constantes quando sabemos que elas serão usadas economiza registradores e torna as operações mais rápidas e eficientes.

As operações que usam imediatos geralmente possuem o nome com o sufixo **i**. Ex.: **addi**.

Operações lógicas:

Em MIPS, operações lógicas são utilizadas para a manipulação de bits, como, por exemplo: Shift Logical Left (**sll**), responsável por deslocar o valor um bit x espaços à esquerda (multiplicação rápida por potência de base 2, em detrimento do mul) e Shift Logical Right (**srl**), responsável por deslocar o valor de um bit x espaços à direita (divisão rápida por potência de base 2).

- **sll** x, y, n # $x \leftarrow y * 2^n$
- **srl** x, y, n # $x \leftarrow y / 2^n$

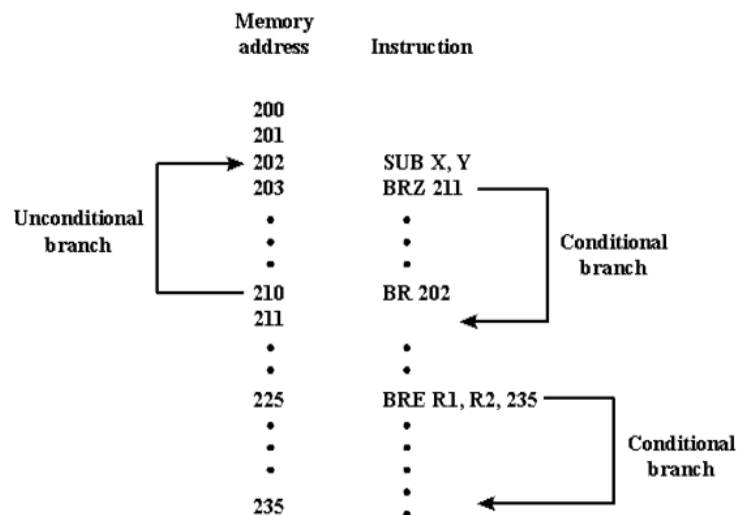
Transferência de controle:

Motivos para usar a transferência de controle:

- É essencial poder executar cada instrução mais do que uma vez (**while**);
- Qualquer programa envolve uma tomada de decisão (**if**);
- Ajuda se houver como dividir a tarefa em pedaços menores (**funções**).

As operações mais comuns de transferência de controle são:

- Desvio (branch);
- Salto (skip);
- Chamada de procedimento.



Operações condicionais:

Desviar (branch) para um label se a condição for verdadeira, se não, continuar sequencialmente:

- **beq** (branch equal) rs, rt, L1 # if (rs == rt), desviar para L1
- **bne** (branch not equal) L1 # if (rs != rt), desviar para L2
- **j** (jump) L1 # **sempre** desviar para L1

label: nome dado a um endereço de memória para facilitar a escrita de código (o assembler do MIPS se encarrega de carregar os endereços dos labels).

Vejamos um exemplo da conversão do código em C: **if (i==j) f = g+h; else f = g-h;** para MIPS, usando como base a tabela de registradores a seguir:

f	\$s0
g	\$s1
h	\$s2
i	\$s3
j	\$s4

1. **bne** \$s3,\$s4,Else # Se \$s3 e \$s4 forem diferentes, pula para Else
2. **add** \$s0,\$s1,\$s2 # Se a instrução anterior não saltou, executa a soma
3. **j** Exit # Se executou a soma, pula para Exit

4. **Else:** `sub $s0,$s1,$s2` # Se \$3 e \$4 eram diferentes, pulou para o trecho com a label Else e executou a subtração
5. **Exit:** ...

Compilando um WHILE:

Vamos converter o código em C: **while (save[i] == k) i += 1;** para MIPS para vermos como isso funciona na prática, usando como base a tabela de registradores a seguir:

i	\$s3
k	\$s5
&save	\$s6

1. **Início:** `sll $t0,$s3,2` # $t0 = i * 2^2 = i * 4$
2. `add $t0,$s6,$t0` # $t0 = \text{save} + i * 4$ (save[i] -> endereço de memória)
3. `lw $t1, 0($t0)` # $t1 = \text{save}[i]$ (valor guardado em $t0$, não o endereço)
4. `bne $t1,$s5,Exit`
5. `addi $s3,$s3,1`
6. `j Inicio`
7. **Exit:** ...

Explicando melhor a lógica utilizada, o primeiro passo que fazemos é obter o valor de `save[i]` para que possamos compará-lo com `k`. No entanto, como não sabemos o valor de `i`, precisamos fazer uma operação mais árdua para conseguir seu valor.

Nas duas primeiras linhas temos o cálculo de endereço de `save[i]`. Como o MIPS é endereçado por bytes e um inteiro (word) ocupa 4 bytes, o índice `i` precisa ser multiplicado por 4 para encontrar o **deslocamento em bytes** correto (o valor de `i + 4` bytes). Deslocar 2 bits para a esquerda (2^2) é a forma mais rápida de se multiplicar por 4. Por exemplo: se `i = 3`, o deslocamento é $3 * 4 = 12$ bytes a partir do início do array. Depois que calculamos o deslocamento na primeira linha, na segunda nós somamos o endereço base do array com o deslocamento que calculamos, ou seja, agora `$t0` contém o endereço de memória exato onde `save[i]` está.

Depois de calculado o deslocamento e achado o endereço de memória de `save[i]`, precisamos obter o valor associado a ele para podermos compará-lo, e fazemos isso usando um `lw` que buscará o dado do endereço de memória `$t0` com um deslocamento de 0 (já que já temos o endereço exato da memória) e o armazenará em `$t1`.

Agora que temos tudo pronto, fazemos a comparação: se `$t1` (valor de `save[i]`) for diferente de `$s5` (`k`), pulamos para o final. Se não, fazemos o incremento de uma unidade usando a adição com imediato no valor de `i` e fazemos um pulo para o início do código, que fará com que o valor de `save[i]` seja obtido novamente para que seja calculado.

Mais operadores condicionais:

Outros operadores condicionais muito úteis são os que comparam se um valor é menor que outro. Neles, definimos o resultado como 1 se a condição for verdadeira e, se for falsa, como 0. São eles:

- **slt (less than)** a, b, c # if (b < c) a = 1; else a = 0;
- **slti (less than immediate)** a, b, const #if (b < const) a = 1; else a = 0;

Se usarmos juntamente com beq e bne, temos:

slt \$t0, \$s1, \$s2 # if (\$s1 < \$s2)

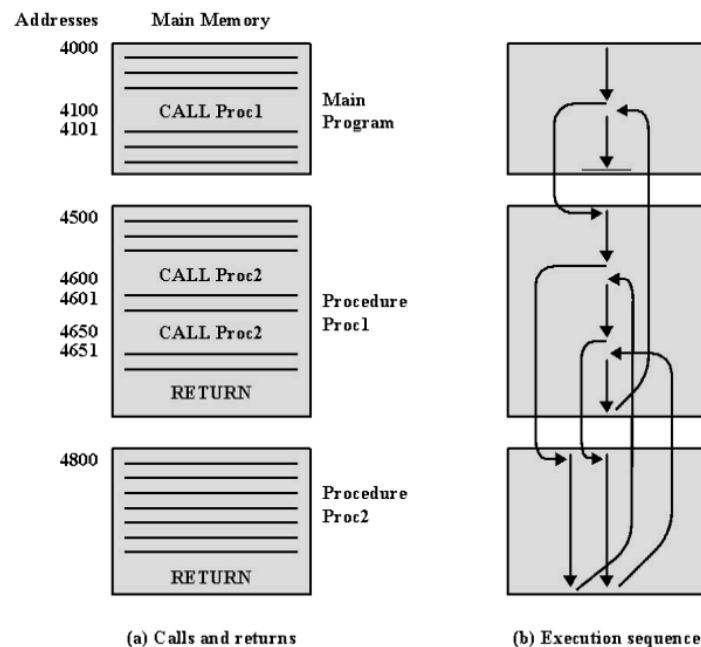
bne \$t0, \$zero, L # se o resultado anterior foi falso (0) (\$s1 >= \$s2), desvia para L

Chamadas de procedimento:

Programas menores incorporados em programas maiores. O procedimento pode ser “chamado” a qualquer momento. O processador desvia, executa o procedimento e volta.

Existem dois principais motivos para se usar procedimentos: a economia de código, já que uma mesma função será utilizada várias vezes, e a modularidade.

Para que um procedimento possa ser executado, é importante que haja uma “chamada” que desvia o código para o procedimento e um “retorno” que volta para onde o procedimento foi chamado.



Existe um registrador específico para guardar o último endereço de retorno de procedimentos, chamado de **\$ra** (return address); Sempre que um procedimento é chamado, o operador **jal** (jump and link) copia o endereço armazenado no PC (que sempre possui o endereço de memória da próxima instrução referente à linha que está

sendo executada), salva-o no \$ra e define o PC com o endereço de memória do procedimento a ser chamado. A partir disso, o procedimento será executado até encontrar um ponto de retorno, devolvendo então o endereço armazenado no \$ra e o copia de volta para o PC, que faz com que o código principal volte a ser executado.

No entanto, se seguirmos esse fluxo sem nenhum recurso extra e tivermos a chamada de um procedimento dentro de outro procedimento, o endereço de retorno da primeira chamada será perdido no momento em que o segundo procedimento for acionado. Por isso, a solução que temos é armazenar o valor dos registradores na memória e acessá-los quando for preciso.

Os passos essenciais para chamadas de parâmetros ocorrerem de forma adequada são:

1. Guardar os parâmetros em registradores;
2. Desviar para o procedimento;
3. Salvar os registradores utilizados;
4. Executar as operações do procedimento;
5. Colocar resultado em registrador;
6. Restaurar os registradores originais;
7. Retornar.

Convenção de registradores

Para manter um padrão de uso dos registradores, existe uma convenção que define o que cada conjunto de registradores pode receber como dados:

- **\$a0 – \$a3**: argumentos;
- **\$v0 e \$v1**: resultados;
- **\$t0 – \$t9**: temporários;
 - Os registradores temporários podem ser sobrescritos e ter seu valor alterado após a chamada de um procedimento.
- **\$s0 – \$s7**: salvos;
 - Os registradores salvos devem ter seu valor guardado e restaurado, ou seja, se a função principal chamou, é dever da função chamada manter seus valores ou, caso ela precise alterá-los, salvá-los na memória e restaurá-los ao final de sua execução.
- **\$sp**: ponteiro de pilha (stack pointer);
- **\$ra**: endereço de retorno (return address).

Instruções de procedimentos

- **Chamada de procedimento**: jump and link (jal)
 - **jal ProcedureLabel** -> o endereço da próxima instrução é salvo em \$ra e o fluxo de execução é desviado para o endereço do label.

- **Retorno de procedimento:** jump register (jr)
 - jr \$ra -> desvia o fluxo de execução para o endereço em \$ra.

Procedimento folha

Procedimentos folha são aqueles que NÃO chamam nenhuma função. Vejamos como ficaria a conversão desse código em C para MIPS:

Código em C:

```
int leaf(int g, int h, int i, int j){
    int f;
    f = (g + h) - (i + j)
    return f;
}
```

Conversão:

```
Leaf:  add $t0,$a0,$a1
       add $t1,$a2,$a3
       sub $v0,$t0,$t1
       jr $ra
```

int g	\$a0
int h	\$a1
int i	\$a2
int j	\$a3

Sua chamada é feita da seguinte forma: jal Leaf

Procedimentos não-folha:

Procedimentos não-folha são aqueles que chamam outros procedimentos durante sua execução. Para fazê-los, devemos salvar os seguintes dados na pilha:

- Seu endereço de retorno (\$ra);
- Registradores salvos que for usar;
- Quaisquer temporários e argumentos que precisar depois;
- Restaurar a pilha.

Vejamos um exemplo de código em C e sua conversão para MIPS:

Código em C:

```
void calcula (int *IN, int *OUT, int n) {
    for (i=0; i<n; i++) {
        OUT[i] = leaf (
            IN[4*i+0], IN[4*i+1],
            IN[4*i+2], IN[4*i+3]);
    }
}
```

Conversão:

```
calcula:
    addi $sp, $sp, -20
    sw    $s0, 0($sp)
    sw    $s1, 4($sp)

    sw    $s2, 8($sp)

    sw    $s3, 12($sp)
    sw    $ra, 16($sp)

    add    $s0, $a0,
    $zero
    add    $s1, $a1,
    $zero
    add    $s2, $a2,
    $zero
    add    $s3, $zero,
    $zero
    sll    $s2, $s2, 2

Loop:
    slt    $t0, $s3, $s2
    beq    $t0, $zero,
    Exit
    sll    $t1, $s3, 2
    add    $t1, $s0, $t1
    lw     $a0, 0($t1)

    lw     $a1, 4($t1)

    lw     $a2, 8($t1)

    lw     $a3, 12($t1)

    jal    leaf
    add    $t1, $s1, $s3

    sw     $v0, 0($t1)

    addi   $s3, $s3, 4
    j      Loop

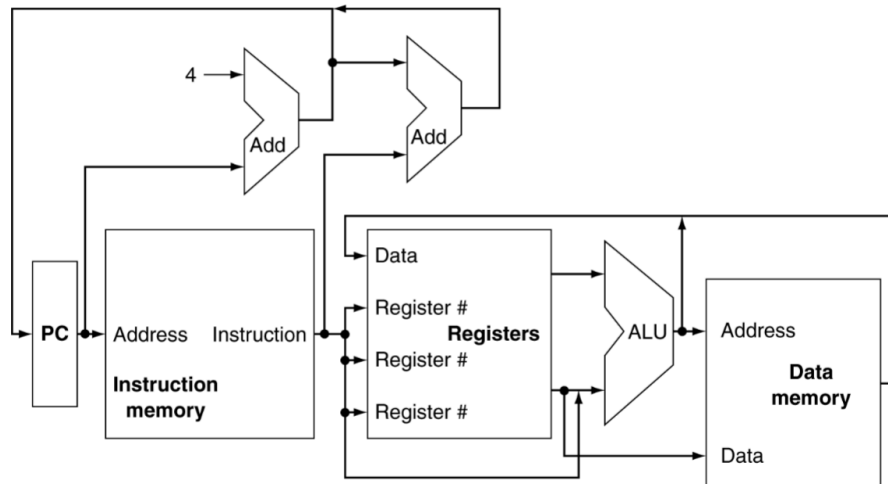
Exit:
    lw     $s0, 0($sp)
    lw     $s1, 4($sp)
    lw     $s2, 8($sp)
    lw     $s3, 12($sp)
    lw     $ra, 16($sp)
    addi   $sp, $sp, 20
    jr     $ra
```

05 - Estrutura da CPU

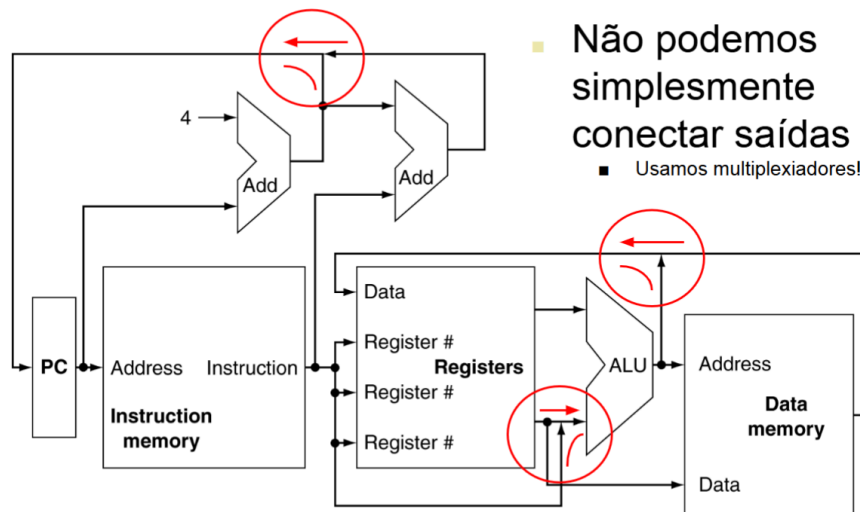
Execução de instruções

- PC → busca instrução na memória de instruções;
- Registradores → busca do banco de registradores;
- Dependendo da instrução:
 - Usa a ALU para calcular:
 - Resultado aritmético;
 - Endereço de memória para load/store;
 - Endereço de destino de um desvio.
 - Acessa a memória para load/store;
 - PC → endereço do desvio ou PC + 4 (PC + 4 bytes → o tamanho de uma instrução).

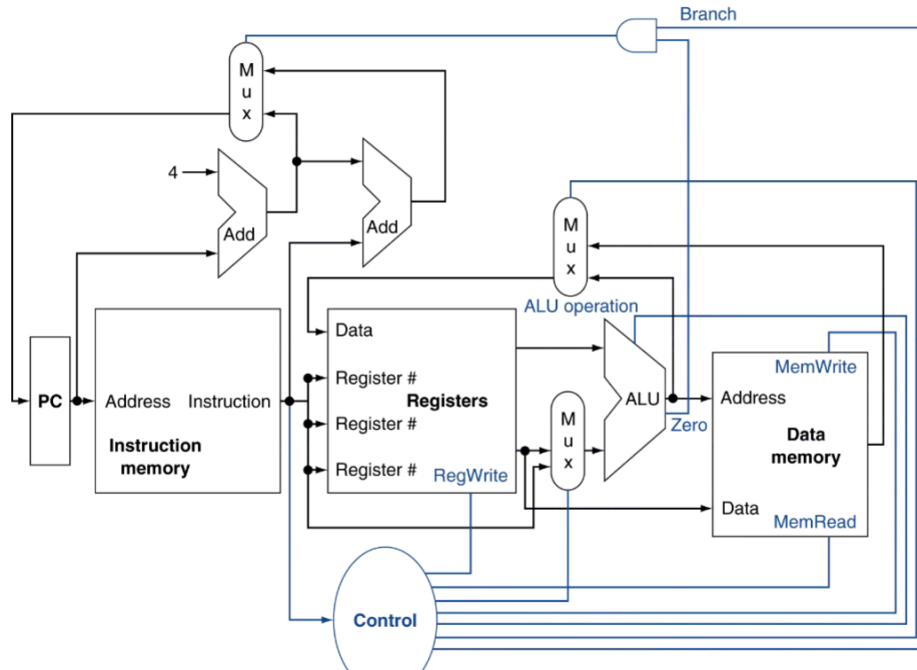
Visão geral de uma CPU



Perceba que nessa visão, temos lugares onde circuitos se encontram e podem ter interferências. Para resolver isso, usamos **multiplexadores** nos locais onde essas informações se cruzam.



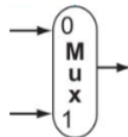
Vejamos agora como realmente é a visão geral de uma CPU com multiplexadores e uma unidade de controle:



Design lógico básico

- Informação codificada em binário:
 - Baixa tensão = 0, Alta tensão = 1;
 - Um fio por bit;
 - Dados multi-bit/byte codificados em barramentos multifios.
- Elementos combinacionais:
 - Operam em dados;
 - Saída é função das entradas.

Multiplexador



○

ALU



- Elementos de estado:
 - Armazenam informações.

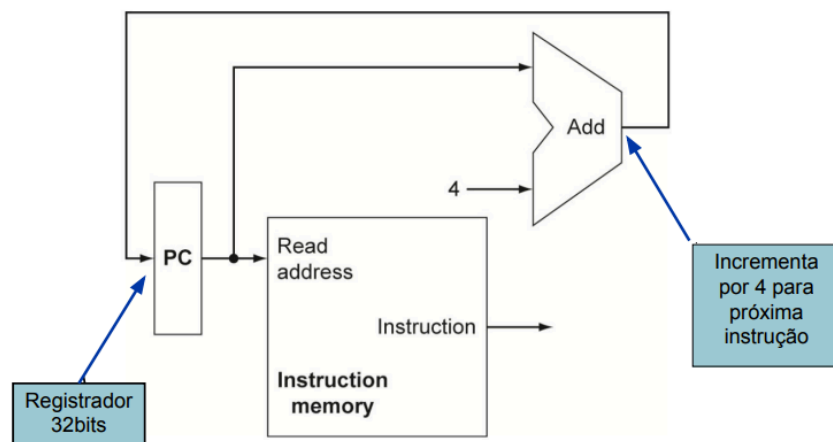
Construindo o Datapath

Datapath é o caminho que uma instrução faz na CPU do momento que ela chega ao momento que ela sai, indicando os elementos que processam dados e endereços na CPU (registradores, ALUs, multiplexadores, memórias, etc.).

A seguir vamos construir, incrementalmente, o datapath de uma versão simplificada do MIPS32 que suporta as seguintes instruções: add, sub, lw, sw e beq.

Busca/Fetch de instrução:

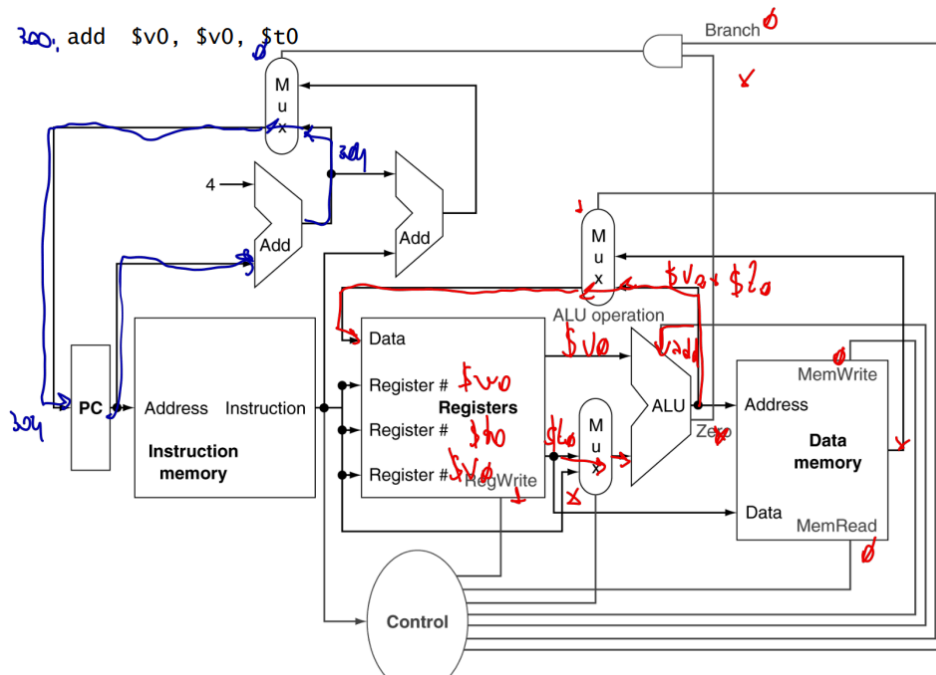
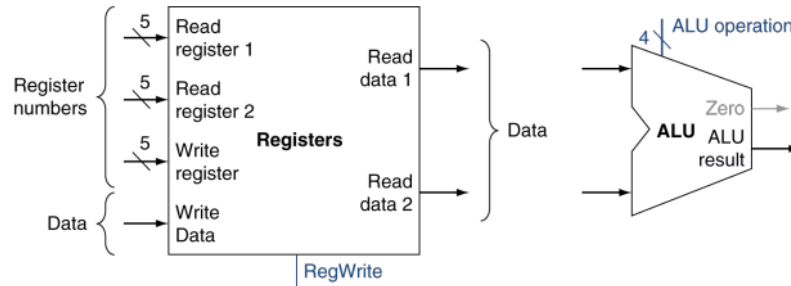
A primeira tarefa que uma CPU realiza quando vai executar uma instrução é obtê-la. Para isso, o PC passa o endereço de memória recebido para a memória de instrução e segue seu caminho para a ALU responsável por incrementar 4 bytes em seu sinal para que ele possa obter a próxima instrução e repassá-la ao PC no próximo ciclo.



Instruções em formato R

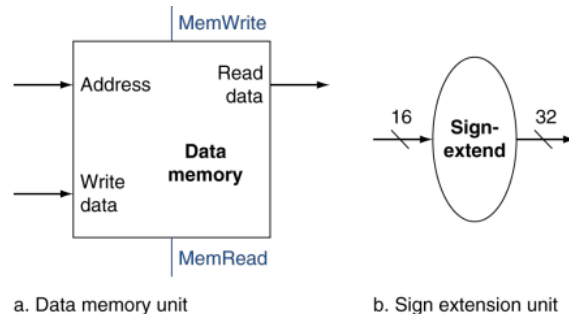
Para processar instruções lógicas/aritméticas, como add, sub, and, or, etc., a CPU passa pela memória de registradores, onde lê dois registradores, realiza a operação lógica/aritmética na ALU e guarda o resultado (write data) em um registrador (que deve indicar o sinal 1 no RegWrite, já que haverá escrita de dados no registrador).

Explicando melhor, depois que saímos da memória de instrução, chegamos nos registradores, onde temos 3 entradas de registradores (2 para os registradores que terão seus dados lidos e 1 para o registrador que receberá o resultado da operação) e uma entrada para o dado que será gravado no registrador de resultado. Do registrador, temos uma saída para o controlador que recebe sinal 0 sempre que houver a escrita de dados em um registrador, e duas saídas que enviam os dados lidos dos registradores para a ALU. Na ALU, o resultado é processado e, se der 0, o fluxo segue pela saída 0 com sinal 1, se não, segue pela saída com o resultado obtido. Se houver escrita na memória, segue para a memória, senão, volta para os registradores pelo campo de dados.



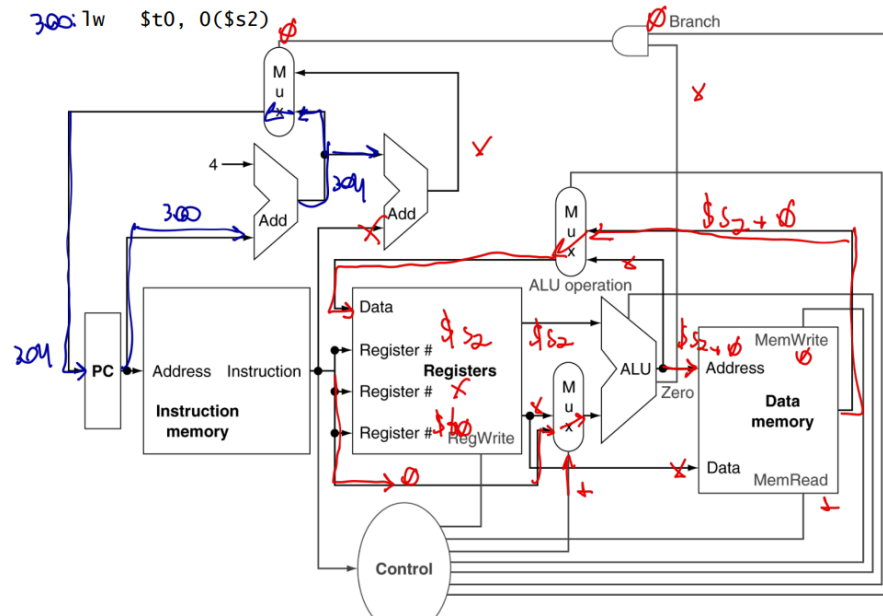
Instruções de Load/Store

Lê operandos dos registradores, sendo necessário calcular o endereço usando offset de 16 bits (4 bytes) usando a ALU.

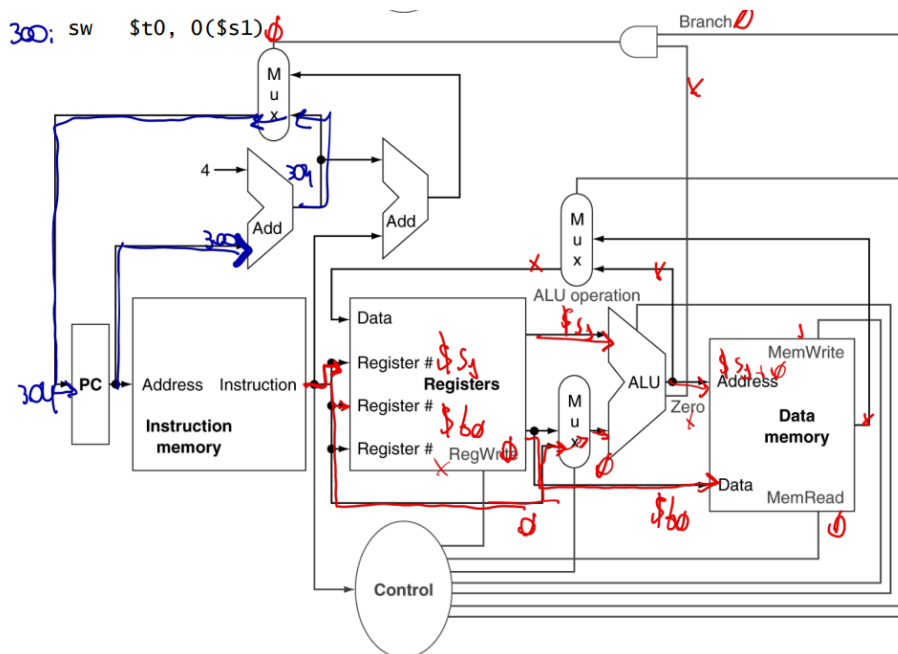


Quando temos Load, **lemos da memória e atualizamos o registrador**. Um dos registradores vai para a ALU para que o endereço da memória seja calculado, enquanto o outro sinal passa como 0 até o multiplexador, que pega o caminho inferior até a ALU até que ambos culminem no endereço de memória. Como é leitura,

MemWrite recebe 0 e MemRead recebe 1, e o resultado da memória de dados volta para o campo de dados no banco de registradores (passando pela parte superior do multiplexador) onde o novo valor será armazenado.

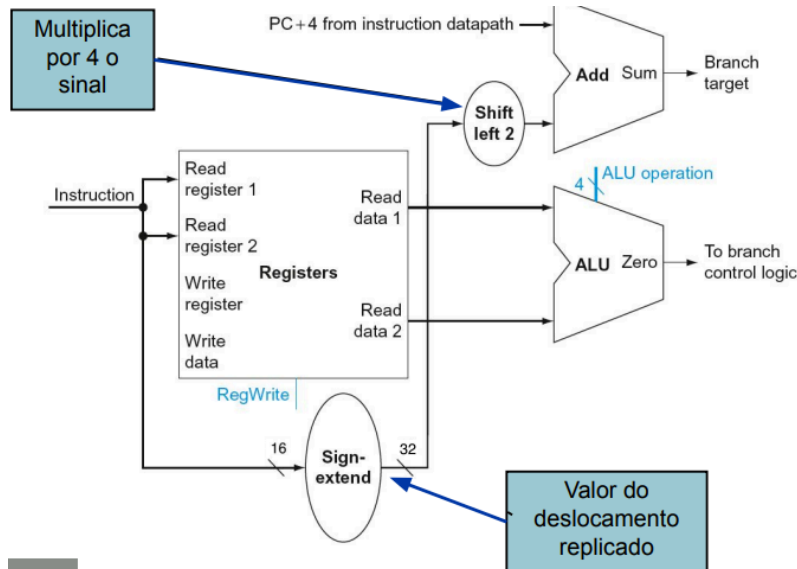


Quando temos Store, **escrevemos de um registrador para a memória**. Um dos registradores vai para ALU para que o endereço de memória seja calculado e o outro vai para o receptor de dados do banco de memória. Como é escrita, MemWrite recebe 1 e MemRead recebe 0, e o fluxo se encerra aí.

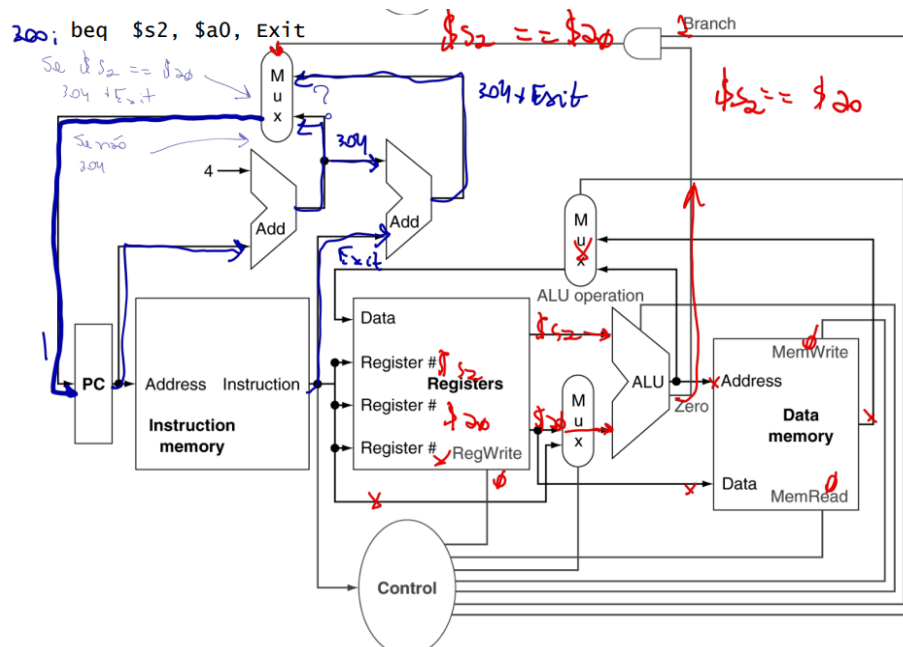


Instruções de Salto:

Lê operandos dos registros e os compara, usando a ALU, subtraindo os valores e checando se a saída é 0. Depois, calcula o endereço do desvio, estende o endereço do desvio fazendo shift left 2 vezes (multiplica por 4 para somar ao PC+4 que já foi calculado pelo fetch anteriormente).



Quando usamos beq, a memória de instrução envia um sinal para o outro somador de endereço do PC para que ele receba o valor necessário para se deslocar para uma branch, fazendo com que as opções de novos endereços caiam num multiplexador que escolherá qual valor usar a depender do resultado lógico da comparação. Enquanto isso, ambos os registradores de leitura vão para a ALU, que ignora completamente a memória e segue pela saída 0 da ALU em direção à branch, que recebe sinal 1. De lá, o resultado da comparação prossegue até o multiplexador, que recebe sinal 1, e decidirá qual caminho seguir.



Problemas de desempenho

O **caminho crítico** (aquele que percorre todas as estruturas) determina o período do clock. Dentre os datapaths mostrados, o crítico é o de load, já que percorre a memória de instruções, o banco de registradores, a ALU, a memória de dados e depois retorna ao banco de registradores.

Fica inviável alterar o período para instruções diferentes, portanto, a solução é o uso de **pipelines**.

...

06 - Aritmética Binária

Notação Posicional

- Como essa notação vale para toda base numérica, o mesmo é válido para a base binária
 - A única diferença é que as potências tem sempre base 2

$$\begin{array}{c} 11010_2 \\ \swarrow \quad \searrow \quad \swarrow \quad \searrow \quad \swarrow \quad \searrow \\ 1x2^4 + 1x2^3 + 0x2^2 + 1x2^2 + 0x2^0 \end{array}$$

Essa ideia pode ser generalizada para qualquer base numérica desejada.

É importante lembrar que precisamos de símbolos suficientes para representar uma base. Por isso, para bases maiores que 10, precisaremos de símbolos além daqueles que já conhecemos como os numéricos.

Conversão binário-decimal

- Acabamos de ver como fazer
- Basta utilizar a regra de numeração posicional

$$10_2 \quad \longrightarrow \quad 1x2^1 + 0x2^0$$

$$1101_2 \quad \longrightarrow \quad 1x2^3 + 1x2^2 + 0x2^1 + 1x2^0$$

Conversão decimal-binário

- Há duas maneiras de fazer essa conversão
- A primeira envolve acumular potências de dois (menores que o valor) até representar o número

$$\begin{array}{ccc} 53_{10} & \longrightarrow & 32 + 16 + 4 + 1 \\ & & \downarrow \\ & & 2^5 + 2^4 + 2^2 + 2^0 \\ & & \downarrow \\ & & 110101 \end{array}$$

A base 16 - Hexadecimal

- Muitas vezes, representar bytes é cansativo e ocupa muito espaço
- Nem sempre os valores são grande a ponto de representarmos com prefixos
- Utilizamos uma base numérica maior (hexadecimal)
- Precisaremos emprestar símbolos do alfabeto, visto que só temos dez símbolos numéricos na base 10

0 1 2 3 4 5 6 7 8 9 a b c d e f

dezesseis símbolos

Conversão hexadecimal - decimal

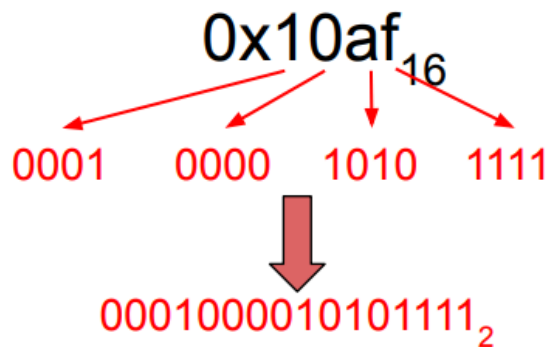
- Basta utilizar a notação posicional

$$0x10af_{16} \longrightarrow 1 \times 16^3 + 0 \times 16^2 + 10 \times 16^1 + 15 \times 16^0$$

ATENÇÃO: costumamos preceder valores hexadecimais pelo prefixo **0x**

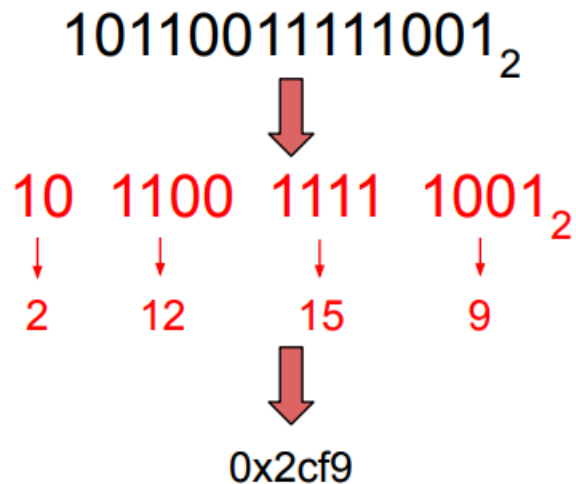
Conversão hexadecimal - decimal **direta**

- Considere a seguinte ideia:
 - Um algarismo hexadecimal armazena valores de 0 a 15
 - Isso corresponde a 4 bits (meio byte)
 - Basta representar cada algarismo hexadecimal como um binário de 4 bits



Conversão binário - hexadecimal

- Basta inverter o processo da conversão hexadecimal -> binário
 - A cada quatro algarismos, contabilize um hexadecimal



Soma - Em Decimal e Binário

- Assim como muito do que vimos anteriormente, podemos estender essa ideia para outras bases, como a base binária
- Na base binária, o cálculo é muito mais simples
- Estamos limitados a dois símbolos, e portanto so há quatro possibilidades
 - $0 + 0 = 0$
 - $0 + 1 = 1$
 - $1 + 0 = 1$
 - $1 + 1 = 0 \dots$ e "vai" 1  10_2

Soma - Em Decimal e Binário

$$\begin{array}{r} 1 \\ + 15 \\ + 7 \\ \hline 22 \end{array} \quad \longrightarrow \quad \begin{array}{r} 1 \ 1 \ 1 \ 1 \\ + 1111 \\ + 111 \\ \hline 10110 \end{array}$$

Subtração em Binário

- Assim como na soma, vamos replicar o mesmo algoritmo adaptado para a base 2
 - $0 - 0 = 0$
 - $1 - 0 = 1$
 - $1 - 1 = 0$
 - $0 - 1 = 1 \dots$ e "empresta" $10_2 = 2_{10}$

Subtração em Binário

$$\begin{array}{r} 1 \ 2 \\ 0 \ 2 \ 0 \ 2 \\ \cancel{10}10011 \\ - 11001 \\ \hline 0111010 \end{array}$$

Multiplicação Binária

- Sem nenhuma surpresa, o mesmo padrão dos exemplos anteriores
- Em base 10, o algoritmo de multiplicação necessita da soma como passo intermediário
 - Em base 2, isso também ocorre
 - $0 \times 0 = 0$
 - $1 \times 0 = 0$
 - $0 \times 1 = 0$
 - $1 \times 1 = 1$

Multiplicação Binária

$$\begin{array}{r} 1011 \\ \times 110 \\ \hline 10000 \\ + 1011 \\ \hline 100010 \end{array}$$

Representação sinal-magnitude

- Sacrificamos o bit mais significativo para representar o sinal do número
 - LSB (least significant bit) - menos significativo/unidade
 - MSB (most significant bit) - mais significativo
- MSB = 0 -> positivo
- MSB = 1 -> negativo

Representação Sinal-Magnitude



Complemento a 1

- Utilizaremos o complemento de um número para representar esse número com valor negativo

$$\begin{array}{c} n \quad 00001100 \quad (12) \\ \downarrow \\ \bar{n} \quad 11110011 \quad (-12) \end{array}$$

- repare: basta inverter todos os bits!

Complemento a 1

- Agora ficou muito mais simples somar dois números inteiros: basta usar a soma binária comum
 - é preciso considerar o bit extra como *carry back* e somá-lo na unidade

$$\begin{array}{r} 16 \\ - 7 \\ \hline 9 \end{array} \quad \rightarrow \quad \begin{array}{r} \overset{1}{} \overset{1}{} \overset{1}{} \\ + \quad 00010000 \quad (16) \\ \quad 11111000 \quad (-7) \\ \hline \quad 00001000 \\ + \quad \\ \overset{1}{} \text{ carry back} \\ \hline \quad 00001001 \end{array}$$

Complemento a dois

- Para solucionar os problemas anteriores, surge o complemento a 2
- Dado um número binário n , seu complemento a 2 n_{c2} é dado por:

$$n_{c2} = \bar{n} + 1$$

- ou, somar 1 no complemento a 1

Obtendo o valor do complemento a dois

- Há duas maneiras:
 - Método 1**
 - Gerar o complemento a 1 do número
 - Somar 1 no valor do complemento
 - Método 2**
 - Partindo do LSB até o MSB, procurar o primeiro bit 1 do número
 - Inverter todos os bits **após** o bit encontrado

Obtendo um valor em complemento a dois

00110100 ₍₅₂₎



11001100 ₍₋₅₂₎

Somando em complemento a dois

- Somar dois números em complemento a dois é muito simples
 - Basta realizar a soma normalmente!
 - Só é necessário considerar o *overflow*
 - Quando somamos, é possível que o resultado ultrapasse os limites da nossa quantidade de bits
 - Se a soma de dois números positivos é negativa, há *overflow*
 - Se a soma de dois números negativos é positiva, há *overflow*
 - Caso contrário, não há overflow

Overflow

- $00110111_2 + 01000011_2 = 01111010_2$
- $10001111_2 + 01111111_2 = 00001110_2$
- $11111111_2 + 11111111_2 = 11111110_2$
- $00111111_2 + 01100000_2 = 10011111_2$ **OVERFLOW**

07 - Hierarquia de Memória (Cache)

Restrições de projeto da memória de computadores se resumizam em três perguntas: **que tamanho, quão rápido e quão caro?**

Existe uma troca entre capacidade, tempo de acesso e custo:

- Quanto **mais rápida** a memória, **maior o custo** por bit;
- Quanto **maior a capacidade** da memória, **menor o custo** por bit;
- Quanto **maior a capacidade** da memória, **maior é o tempo de acesso**.

Resolvemos esse dilema não focando em apenas um componente ou tecnologia de memória, mas **montando uma hierarquia entre elas**, visando aumentar o leque de opções e tipos para que desfrutemos de várias tecnologias para casos específicos.

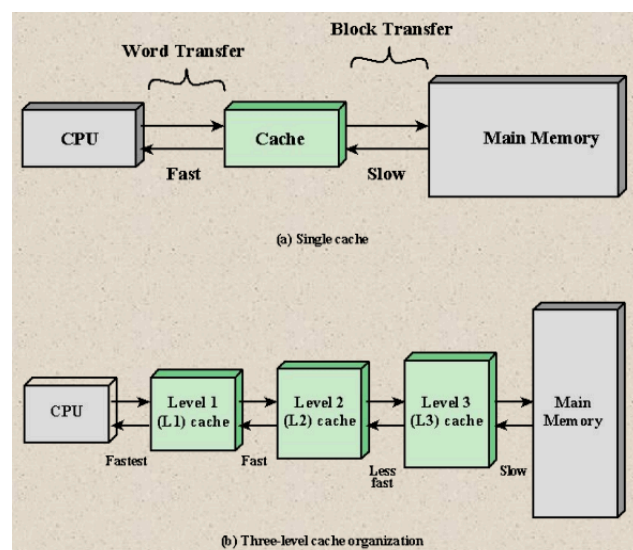
A título de curiosidade, algumas das tecnologias de memórias que existem, o tempo de acesso e o custo associado a elas estão contidos a seguir:

- **Static RAM (SRAM):** de 0,5 ns a 2,5 ns e de US\$5000 a US\$50000 por GB;
- **Dynamic RAM (DRAM):** de 50 ns a 100 ns e de US\$10 a US\$18 por GB (antes do advento da IA, girava em torno de US\$3 a US\$6 por GB);
- **Flash:** 5 μ s a 50 μ s e de US\$0.05 a US\$0.12 por GB;
- **Disco Magnético:** 5 ms a 20 ms e de US\$0.01 a US\$0.02 por GB.

Quando vamos construir um computador, queremos o tempo da SRAM, mas o custo do disco, embora isso não seja possível.

O Princípio da Localidade

Programas acessam uma proporção pequena de toda a memória disponível. **O princípio da localidade temporal** defende que **itens acessados recentemente devem ser acessados logo** (como ocorre quando acessamos instruções em um laço ou usamos variáveis de indução (como controladores de laços)) e se assemelha à ideia de pegar um livro na biblioteca: quando o pegamos, nós o trazemos para nossa mesa e o deixamos próximos de nós, considerando que iremos acessar seu conteúdo repetidas vezes, se contrapondo com a ideia de ir à estante do livro cada vez que precisássemos de uma informação dele. Nesse contexto, nossa mesa é o equivalente à memória cache, que guarda os dados de acesso rápido, enquanto a estante se assemelha ao nosso disco ou RAM. De maneira

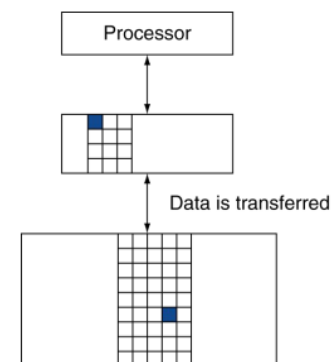


semelhante, o **princípio da localidade espacial** defende que **itens perto daqueles acessados recentemente também devem ser acessados logo**, copiando um bloco ao redor do dado que está sendo buscado ao invés de buscar um dado por vez (como ocorre quando acessamos arrays ou instruções sequenciais, por exemplo).

Quando trabalhamos com a hierarquia de memória, o comportamento padrão é armazenar todos os dados no disco. A partir daí, quando acessamos um dado, copiamos os itens acessados recentemente (e seus vizinhos) do disco para uma memória DRAM menor (a memória principal). No caso de dados acessados mais recentemente ainda, os itens (e seus vizinhos) são copiados para uma memória SRAM menor (a memória cache). O processo de transferência dos dados é ilustrado na figura anterior.

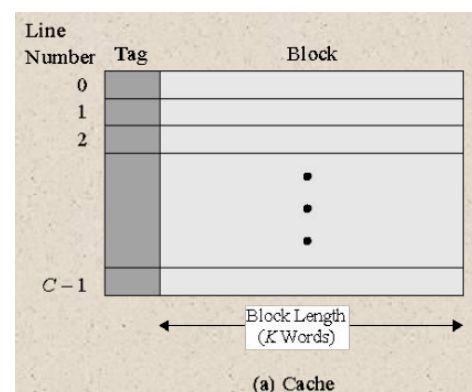
Níveis de Hierarquia de Memória

Bloco (ou linha) é o nome dado ao tamanho do dado que será transferido, podendo ele ser uma ou mais palavras. **Hit** é o nome dado ao processo de encontrar um dado em alguma das caches, ou seja, quando o dado acessado está em um nível próximo ao processador e temos o acesso concretizado nesse nível (a taxa de Hits é dada pela fórmula **hits/acessos**). Toda vez que não encontramos um dado em qualquer uma das caches, ou seja, a linha que estamos buscando está em um nível mais distante, chamamos isso de **Miss**. Quando temos um miss, imediatamente copiamos o bloco desejado no nível mais próximo da cache. Um miss faz com que o acesso a um dado seja mais lento e a taxa de Miss é dada pela fórmula **misses/acessos** ou **1 - taxa de hits**). Em caches com vários níveis, temos a L1, que fica junto da CPU, a L2, que lida com misses da L1 e a L3 e L4 ou a DRAM responsáveis por lidar com os misses da L2.



Bits de Tag e Validade

Como é possível saber qual dado está em um bloco de cache? Simples! Isso é feito guardando o **endereço** na DRAM **junto com o dado**. Fazendo isso, só precisamos dos **bits mais significativos**, chamados de **tags**. Mas e se aquela linha não tiver dados? Aí usamos os **bits de validade**, que assumem valor **1** quando **há dados** e **0** (o valor inicial) quando **não há dados**.



Misses na Cache

Em um **hit** na cache, a **CPU prossegue normalmente** com suas operações, uma vez que obteve com sucesso o dado que estava buscando. No entanto, **quando um miss ocorre**, a CPU precisa **parar o pipeline** (inserir bolhas no pipeline), **buscar a linha de cache no próximo nível** e, se for uma instrução, reiniciar sua busca, caso seja um dado, apenas completa seu acesso a ele.

Função de Mapeamento

A memória principal é dividida logicamente em unidades de tamanho fixo chamadas **blocos**, enquanto a cache é dividida principalmente em espaços de mesmo tamanho chamados **linhas** (ou slots). Como existem muito mais blocos na memória principal do que linhas na cache, a **função de mapeamento** define as regras de como e onde um bloco de memória poderá ser alocado quando for trazido para a cache. Para isso, existem 3 técnicas de busca que usam o endereço binário gerado pela CPU de forma interpretada pela cache como uma divisão de campos específicos:

Mapeamento Direto:

No mapeamento direto de Cache, se todas as vagas estão ocupadas, o dado a ser usado substitui o lugar de uma delas. É a técnica mais simples, uma vez que cada bloco da memória principal possui mapeamento fixado para **apenas uma única linha possível** de cache. A linha alvo i é calculada por meio da seguinte operação:

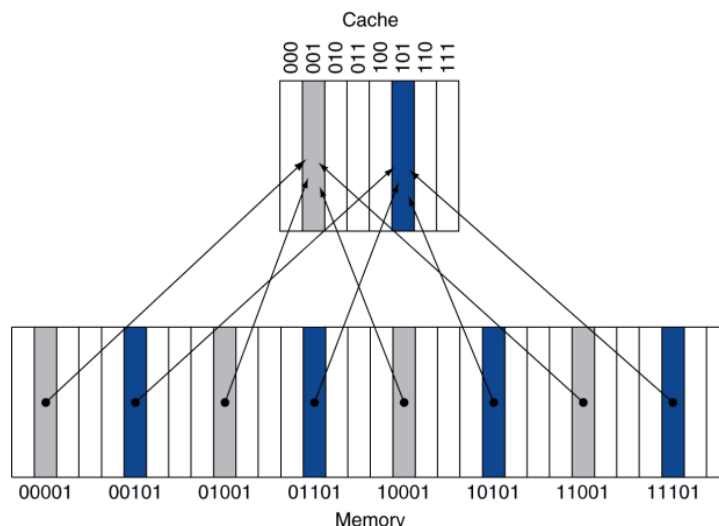
$i = j \pmod{m}$, onde i é o número da linha da cache, j é o número do bloco na memória principal e m é o número total de linhas disponíveis na cache.

O **endereço da memória principal é dividido em 3 campos distintos** para a busca de um dado: **tag** (os bits mais significativos, que identificam em qual bloco específico da memória principal está armazenado na linha da cache), **conjunto/índice/linha** (bits do meio que determinam em qual linha da cache o bloco deve obrigatoriamente residir) e **palavra/offset** (aponta para o byte ou palavra exata dentro da linha).

Sua principal **vantagem** é o **baixo custo e complexidade de implementação**, além do fato de a **verificação de acerto (hit) ser instantânea**, uma vez que só precisa comparar uma tag.

Sua **desvantagem**, por outro lado, é o **alto índice de misses por conflito**. Se um programa alternar acessos repetidos a dois blocos de memória diferentes que mapeiam a mesma linha da cache, eles ficarão se expulsando continuamente (fenômeno conhecido como thrashing).

Voltando a falar sobre a localização dos blocos nas linhas da cache, nesse modelo, só existe um local possível para dados de um mesmo índice e que é determinado pela fórmula: **(endereço do bloco) % (nº de blocos na cache)**, sendo o nº de blocos sempre uma potência de 2. O índice de cada bloco é determinado pelos bits menos significativos do conjunto mais significativo. No exemplo da tabela a seguir em que temos o endereço binário **10110**, 10 é a **tag** e 110 é o **índice**.



Agora vejamos um exemplo de uma cache de 8 linhas, 1 palavra/linha, diretamente mapeada e cujo índice é o número da linha, assim como a tabela indicando as chamadas de acesso aos dados contidos nos endereços indicados:

Endereço	Hit/miss	Bloco
22 10 110		
26 11 010		
22 10 110		
26 11 010		
16 10 000		
3 00 011		
16 10 000		
18 10 010		

Index	V	Tag	Dado
000			
001			
010			
011			
100			
101			
110			
111			

Se preencheremos de acordo com as especificações, o resultado obtido na tabela final será o seguinte, considerando a substituição do dado do endereço 16 pelo 18:

Index	V	Tag	Data
000	Y	10	Mem[10000]
001	N		
010	Y	10	Mem[10010]
011	Y	00	Mem[00011]
100	N		
101	N		
110	Y	10	Mem[10110]
111	N		

Além disso, a tabela de Hit/miss seria preenchida, respectivamente, por: **Miss - Miss - Hit - Hit - Miss - Miss - Hit - Miss**.

Vale ressaltar que o valor contido no bloco é, nesse caso, o mesmo do índice.

Para finalizarmos, vamos nos passar brevemente pelos conceitos que abrangem tamanhos de blocos maiores que os padrões de 32 bytes. No geral, blocos maiores deveriam reduzir a taxa de miss, seguindo os princípios da localidade espacial, mas se apenas os blocos aumentam e a cache permanece do mesmo tamanho, essa taxa, na verdade, aumenta, visto que, quanto maiores os blocos, menos blocos estarão disponíveis e, conseqüentemente, mais blocos disputarão por uma linha na cache (devido à menor variedade de blocos possíveis de armazenar), culminando no aumento inevitável da taxa de miss.

Mapeamento Associativo:

No mapeamento associativo (n-way), as vagas estão “reservadas” e a substituição de dados passa por um processo mais complexo de decisão. Ela permite que qualquer bloco seja mapeado em qualquer linha vaga da cache, garantindo flexibilidade total para o armazenamento. Nesse modelo, **não existe campo de “Linha/index”**, uma vez que o **endereço binário é dividido unicamente em tag** (bits significativos contendo a identidade do bloco) **e word/offset** (posição interna do dado dentro do bloco). Para encontrar um dado, o controle deve comparar todas as Tags na cache até que encontre o bloco com a Tag desejada.

Entre as **vantagens** desse modelo, vale destacar a **eliminação completa de misses por conflito**, visto que uma linha só é desalocada quando a cache inteira estiver completamente lotada.

Por outro lado, a **desvantagem** desse modelo é justamente que, para determinar se um dado está na cache, **o circuito de controle precisa varrer e comparar a tag do endereço gerado com todas as tags de todas as linhas** da cache simultaneamente. Isso exige circuitos comparadores complexos em paralelo para cada entrada, aumentando o custo do hardware e elevando o consumo de energia.

Devido ao custo elevado, esse tipo de mapeamento é restrito a estruturas pequenas e ultra criticamente rápidas, como buffers de tradução de paginação virtual (TLBs).

Mapeamento Associativo por Conjuntos:

O mapeamento associativo por conjuntos funciona como uma mistura das duas técnicas anteriores (abordagem híbrida), fazendo uso das vantagens de cada uma para diminuir suas desvantagens gerais. Nele, a cache é subdividida em **v** estruturas menores denominadas **conjuntos** (sets). Cada conjunto abriga uma quantidade fixa de **n** linhas (conhecida como cache n-way set-associative) e o bloco de memória é mapeado de forma direta para um conjunto específico através da mesma fórmula do mapeamento direto ($i = j \pmod{v}$). Dentro do conjunto indicado, a busca é feita de

Nesse mapeamento, o endereço é dividido da mesma forma que no mapeamento direto (com o index definindo qual conjunto será acessado) e a flexibilidade do sistema depende do número de linhas alocadas por conjunto (k-way). Por exemplo: em um MAPC (mapeamento associativo por conjunto) de 2 vias (2-way), cada conjunto possui 2 linhas que serão verificadas e terão os dados comparados simultaneamente pelo processador. Em um MAPC de 4 vias (4-way), cada conjunto tem 4 linhas disponíveis, o que oferece maior flexibilidade e reduz as chances de miss de conflito.

Atualmente esse modelo de mapeamento é o padrão absoluto da indústria de computadores modernos para o projeto das caches L1, L2 e L3 (por exemplo, configurações comuns de caches de dados de processadores Intel Core ou AMD Ryzen mapeadas em 8-way ou 16-way).

One-way set associative (direct mapped)

Two-way set associative

Four-way set associative

Eight-way set associative (fully associative)

[illegible]

Algo interessante de ser notado é a diminuição da taxa de miss com o aumento da associatividade. Embora não muito expressiva, se adotarmos como modelo uma cache de 64KB com 16 palavras/bloco, SPEC2000, se usamos o padrão 1-way a taxa é 10.3%, 2-way: 8.6%, 4-way: 8.3% e 8-way: 8.1%.



Quando precisamos decidir qual dado descartar da cache em uma cache associativa, o critério é sempre com base na localidade temporal: o dado que não é acessado há mais tempo deverá ser descartado.

Determinação de quantidade de linhas e Tag, Index e Offset de endereços binários:

Quando queremos determinar quais partes de um endereço correspondem aos componentes necessários da cache, podemos fazer uso das seguintes fórmulas:

- **Offset:** $\log_2(\text{tamanho do bloco})$ → ou seja, se tenho um bloco de 4 bytes, meu offset será de 2 bits, visto que $2^2 = 4$ e é preciso 2 bits para contar até 4 (0 a 3);
- **Index:** $\log_2(\text{quantidade de conjuntos})$ → ou seja, se eu tenho 8 conjuntos, meu index será de 3 bits, visto que $2^3 = 8$ e é preciso 3 bits para contar até 8 (0 a 7);
- **Tag:** $\#bits \text{ do endereço} - \#bits \text{ do index} - \#bits \text{ do offset}$ → ou seja, se meu endereço tem 8 bits e eu reservei 2 para o offset e 2 para o index, minha tag será de 4 bits.

Em um **exemplo prático**, vamos supor que temos uma cache associativa por conjuntos de 2 vias com capacidade de armazenamento de 32 bytes e blocos de 4 bytes. A partir desses dados, podemos determinar que a **quantidade de linhas** disponíveis será de **8 linhas** com **4 bytes** de armazenamento cada (32 bytes de armazenamento total / 4 bytes por bloco de memória armazenado em cada linha). Seguindo nessa mesma lógica, considerando que temos 8 linhas e nossa

associatividade é de 2 vias, concluímos que a **quantidade de conjuntos** será **4** (8 linhas / 2 linhas por conjunto). Isso fará com que nossa cache apresente a seguinte estrutura:

Two-way set associative

Set	Tag	Data	Tag	Data
0				
1				
2				
3				

Aqui, para cada Set (conjunto), temos duas linhas de cache reservadas. Se representássemos a mesma cache de forma vertical, teríamos as 8 linhas empilhadas e cada set englobando conjuntos de 2 linhas.

Continuando a determinação dos conceitos necessários para a resolver um exercício com esses dados, precisamos definir as partes do endereço que serão

usadas para a tag e o índice do conjunto. Como temos um **bloco de memória de 4 bytes**, a parte reservada para o offset será $\log_2(4 \text{ bytes}) = 2 \text{ bits}$, o **index** será $\log_2(4 \text{ conjuntos}) = 2 \text{ bits}$ e a **tag** será 8 bits totais - 2 bits de offset - 2 bits de index = **4 bits de tag**. Portanto, se tivermos o endereço binário **00100100**, o offset será **00** (ou apenas 0), o index será **01** (ou 1 em decimal) e a tag será **0010** (ou 2 em decimal).

A seguir, veremos um exemplo de comparação de uso de caches diretamente mapeadas, totalmente associativas e associativas por conjunto para a inserção da seguinte ordem de blocos de acesso: **0, 8, 0, 6 e 8**.

- **Diretamente Mapeada:**

Bloco	Index	Hit/miss	Conteúdo da cache após o acesso			
			0	1	2	3
0	0	miss	Mem[0]			
8	0	miss	Mem[8]			
0	0	miss	Mem[0]			
6	2	miss	Mem[0]		Mem[6]	
8	0	miss	Mem[8]		Mem[6]	

- **2-way associativa por conjuntos:**

Bloco	Index	Hit/miss	Conteúdo da cache após o acesso			
			0		1	
0	0	miss	Mem[0]			
8	0	miss	Mem[0]	Mem[8]		
0	0	hit	Mem[0]	Mem[8]		
6	0	miss	Mem[0]	Mem[6]		
8	0	miss	Mem[8]	Mem[6]		

- **Completamente associativa:**

Bloco		Hit/miss	Conteúdo da cache após o acesso			
0		miss	Mem[0]			
8		miss	Mem[0]	Mem[8]		
0		hit	Mem[0]	Mem[8]		
6		miss	Mem[0]	Mem[8]	Mem[6]	
8		hit	Mem[0]	Mem[8]	Mem[6]	

Algoritmos de substituição

Se a cache está cheia e um novo bloco deve ser armazenado na cache após um miss, um dos blocos existentes precisa ser substituído. Quando trabalhamos com o mapeamento direto, só existe uma única opção para esta substituição, visto que cada linha é destinada a um índice específico. No entanto, quando trabalhamos com mapeamento associativo ou associativo por conjuntos, é necessária a implementação de **algoritmos de substituição**, que são responsáveis por determinar qual bloco será substituído com base em seus critérios. Inclusive, vale ressaltar que, para que sejam mais rápidos, esses algoritmos são implementados no hardware.

LRU (Least Recently Used):

Ele identifica e substitui o bloco dentro do conjunto que está há mais tempo sem receber nenhum tipo de acesso (seja para leitura ou gravação) por parte do processador. Esse algoritmo baseia-se estritamente na premissa da **localidade temporal**: o bloco que está há mais tempo sem ser acessado é substituído, já que ele provavelmente não será demandado logo.

É o algoritmo mais eficaz e mais empregado na indústria, usado principalmente nos níveis L1 e L2 da cache para maximizar a taxa de hit, apesar de apresentar um custo alto de circuitos por precisar de bits de controle adicionais por linha (contadores de uso ou matrizes lógicas) para atualizar as "idades" relativas dos acessos a cada ciclo de instrução.

FIFO (First-In, First-Out):

Ele organiza logicamente as linhas ocupadas do conjunto como uma fila. O bloco que foi trazido há mais tempo para a cache é selecionado para ser substituído ignorando completamente se ele está ou não sendo muito referenciado no momento atual.

Em relação ao sistema anterior, esse algoritmo é fácil de ser implementado, já que o hardware precisa apenas de um registrador ponteiro circular simples para apontar qual elemento do conjunto será a próxima vítima da substituição. No entanto, esse sistema pode remover dados cruciais apenas pelo fato de terem sido trazidos há mais tempo do que outros. Esse modelo é aplicado principalmente em processadores e controladores de baixo custo, que dispõem de uma área menor para os circuitos de controle do chip.

LFU (Least Frequently Used):

Ele substitui o bloco que foi menos acessado desde o instante em que foi originalmente copiado para o interior da cache. Ou seja, o que tiver menos hits em um determinado período é substituído.

Esse modelo raramente é usado em caches de CPUs comuns, sendo mais utilizado em soluções híbridas de caches lógicas via software (como servidores de BD ou caches proxies web). Sua implementação exige um registrador contador binário acoplado a cada linha de cache que é incrementado sempre que um hit ocorre. No entanto, sofre o problema de “saturação de passado”, já que, se eu fiz muitos hits no passado mas nunca mais o usei, o dado demorará para ser substituído devido ao acúmulo no contador.

Aleatório (Random):

Ele seleciona um slot de linha de forma totalmente aleatória dentro do conjunto afetado quando é necessária a substituição e possui efetividade semelhante ao LRU conforme o tamanho e o grau de associatividade aumentam.

Para implementá-lo, não é necessário nenhum bit de controle extra, já que ele usa apenas um circuito gerador de ruído pseudo aleatório simples conectado à lógica de substituição. Geralmente, esse modelo é usado em caches muito grandes ou de nível superior (L3 massivas e L4) que são compartilhadas por múltiplos núcleos lógicos de processadores ou servidores, onde rastrear os estados individuais de cada linha geraria uma sobrecarga térmica e de consumo elétrico inviáveis.

Em resumo, se tivermos acesso à memória buscando os endereços 0, 0, 0, 0, 0, 8, 5, 6 e 7, no modelo **LRU**, o **0** será substituído; no modelo **Fila**, o **0** será substituído; no modelo **LFU**, **qualquer um menos o 0** poderá ser substituído (talvez seja o 8) e no **Aleatório**, **qualquer um** poderá ser substituído.

Algoritmos de Propagação de Alterações

Write Through:

É a técnica mais simples de propagação de alterações, uma vez que todas as operações de escrita são feitas na memória principal e na cache, ou seja, se alteramos um valor da memória que já está na cache, no momento em que ele é alterado, a memória principal também é alterada. No entanto, a principal **desvantagem** desse sistema é o aumento do tráfego na memória principal, reduzindo o desempenho.

Write Back:

Essa técnica é empregada quando queremos minimizar o número de escritas na memória principal, uma vez que as escritas são feitas somente na cache e a memória principal só será alterada quando o bloco é substituído e sair da cache. No entanto, quanto mais mecanismos de controle, maior o tempo de acesso à cache.

Política de Write Back:

Quando um bloco precisa ser substituído, há dois casos a se considerar: se o bloco antigo **não foi modificado**, ele pode ser sobrescrito sem ser salvo antes no próximo nível; **se houve uma escrita no bloco**, então ele precisa ser enviado para o próximo nível de cache antes de ser substituído.

Além disso, quando usamos esse modelo, há dois problemas a se considerar: mais do que um dispositivo ter acesso à memória principal e a máquina possuir mais de um processador e, todos com uma cache local, estarem compartilhando a mesma memória principal (risco de conflitos ou acesso a dados desatualizados).

Origem de Misses

Misses podem ocorrer por diversos fatores quando fazemos acesso a dados. Para isso, existem classificações para os misses de acordo com aquilo que os origina:

- **Miss compulsório:** primeiro acesso a um bloco;
- **Miss por capacidade:** devido ao tamanho limitado da cache, um bloco que foi substituído é acessado novamente;
- **Miss por conflito/colisão:** em uma cache associativa por conjuntos, devido à competição por entradas em um conjunto, ocorre quando um dado é substituído por outro de mesmo índice e precisa ser acessado novamente.

Desempenho da Cache

O desempenho geral de processamento da CPU está atrelado à taxa de sucesso da cache. Como descrito anteriormente, quando o processador requisita um dado e ocorre um acerto (hit), a execução segue sem atrasos. No entanto, quando ocorre uma falha (miss), o pipeline do processador deve ser paralisado (stalled) até que a linha de cache correspondente seja resgatada no nível de memória inferior, o que acaba consumindo preciosos ciclos de clock adicionais (denominados **penalidade de miss**).

A métrica padrão para avaliar esse impacto baseia-se na equação de tempo de CPU efetivo:

$$\text{CPI Efetivo} = \text{CPI Base} + \text{Stall}_{\text{I-cache}} + \text{Stall}_{\text{D-cache}}.$$

Vamos conferir um exemplo para esclarecer melhor esses conceitos.
Consideremos um sistema computacional real com as seguintes regras:

- **CPI Base (CPI ideal):** 2,0 ciclos por instrução (considerando uma cache perfeita que nunca falha);
- **Taxa de miss de Instrução (Miss I-cache):** 2% dos acessos (0,02);
- **Taxa de miss de Dados (Miss D-cache):** 4% dos acessos (0,04);
- **Penalidade de Miss:** 100 ciclos de clock de espera obrigatórios para buscar na memória DRAM;
- **Frequência de Instruções de memória:** as instruções de leitura e escrita correspondem a 36% do total de instruções de um programa a ser executado.

Para calcularmos o desempenho efetivo, seguimos o seguinte passo a passo:

1. **Cálculo dos Ciclos de Atraso causados pela Cache de Instruções (I-Cache):** como toda e qualquer instrução precisa necessariamente ser buscada na memória antes de ser executada, a taxa de erro de busca se aplica à totalidade do programa:
 - a. Stall I-cache = Taxa de Miss I-cache * Penalidade;
 - b. Stall I-cache = $0,02 * 100 = 2$ ciclos por instrução.
2. **Cálculo dos Ciclos de Atraso causados pela Cache de Dados (D-Cache):** os erros na cache de dados afetam somente as instruções que explicitamente manipulam a memória, portanto:
 - a. Stall D-cache = Freq. load/store * Taxa de Miss D-cache * Penalidade;
 - b. Stall D-cache = $0,36 * 0,04 * 100 = 1,44$ CPI.
3. **Cálculo do CPI Efetivo do Sistema Real:** somamos o tempo base computacional com a somatória de todas as penalidades geradas pelas falhas na cache:
 - a. CPI Efetivo = CPI Base + Stall I-cache + Stall D-cache;
 - b. CPI Efetivo = $2 + 2 + 1,44 = 5,44$ CPI.

A partir disso, podemos concluir que o impacto real sofrido pelo processador é extremamente agressivo: o CPI real saltou de 2 para 5,44. Realizando a razão de desempenho ($5,44 / 2 = 2,72$), conclui-se que o computador real opera de forma 2,4 vezes mais lenta do que operaria caso possuísse uma memória infinitamente rápida. Isso demonstra graficamente por que otimizar caches é vital no projeto de arquiteturas de computadores.

08 - Álgebra booleana

Portas lógicas

Portas/circuitos lógicos são dispositivos que operam e trabalham com um ou mais sinais lógicos de entrada (1 ou 0) para produzir uma e somente uma saída, dependente da função implementada no circuito.

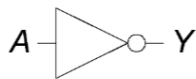
Portas realizam operações lógicas (inversão/NOT, AND, OR, NAND, NOR, etc.), podendo possuir uma entrada (NOT gate, buffer), duas entradas (AND, OR, XOR, NAND, NOR, XNOR) ou múltiplas entradas.

Portas lógicas de uma entrada:

- **NOT:**

O NOT é responsável por fazer a **inversão do sinal**, ou seja, se entra 1, sai 0; se entra 0, sai 1. A **bolinha** após o triângulo é um **indicador de inversão**.

NOT



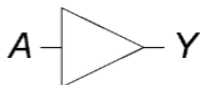
$$Y = \overline{A}$$

A	Y
0	1
1	0

- **BUF:**

O Buffer é um **repetidor de sinal**, ou seja, se entra 1, sai 1; se entra 0, sai 0.

BUF



$$Y = A$$

A	Y
0	0
1	1

Portas lógicas de duas entradas:

- **AND:**

Na porta AND, apenas **duas entradas de 1** (true) **culminam em 1** (true). Se uma ou mais entradas possuírem 0 (false), o resultado é 0 (false).

AND



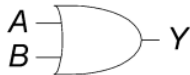
$$Y = AB$$

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

- **OR:**

Na porta OR, se **pelo menos uma das entradas for 1** (true), **a saída é 1** (true). A saída é falsa apenas se ambas as entradas forem 0 (false).

OR



$$Y = A + B$$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

- **XOR:**

Na porta XOR (OR exclusivo), **a saída só é verdadeira se os sinais de entrada são diferentes**, ou seja, se são **1 e 0** ou **0 e 1** (pode ser interpretada como $\bar{A}B + A\bar{B}$).

XOR



$$Y = A \oplus B$$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

- **NAND:**

Na porta NAND (NOT AND), a saída é o **inverso da porta AND**, ou seja, só é **falsa quando ambas as entradas são 1** (true).

NAND



$$Y = \overline{AB}$$

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

- **NOR:**

Na porta NOR (NOT OR), **a saída é 1** (true) somente quando **ambas as entradas são 0** (false).

NOR



$$Y = \overline{A + B}$$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

- **XNOR:**

Na porta XNOR (exclusive NOT OR), **a saída é 1** (true) somente quando os **sinais de entrada são iguais**, ou seja, 0 e 0 (ambos false) ou ambos 1 (true) (pode ser interpretada como $AB + \overline{A}*\overline{B}$).

XNOR



$$Y = \overline{A \oplus B}$$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

Portas lógicas de múltiplas entradas:

As portas lógicas de múltiplas entradas funcionam com a mesma lógica das portas de 2 entradas, mas com a adição de uma ou mais entradas afetando a operação.

- **NOR3:**

Na porta NOR3 (NOT OR de 3 entradas), **a saída é 1** (true) somente quando **todas as 3 entradas são 0** (false).

NOR3



$$Y = \overline{A+B+C}$$

A	B	C	Y
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

- **AND3:**

Na porta AND3 (AND de 3 entradas), **a saída é 1** (true) somente quando **todas as entradas são 1** (true).

AND3

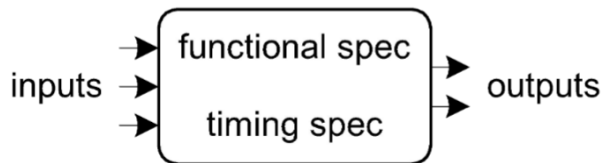


$$Y = ABC$$

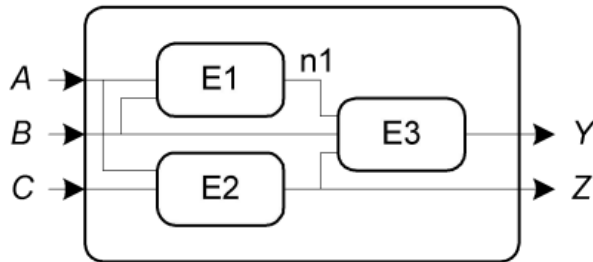
A	B	C	Y
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

Circuitos Lógicos

Circuitos lógicos são compostos de **entradas**, **saídas** e uma **especificação**.

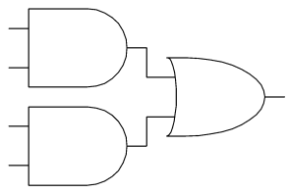


Conforme exemplificado na imagem abaixo, eles são estruturados por sinais de **entrada** (A, B e C), **saída** (X e Y) e **internos** (N1) e são intermediados por **elementos** (E1, E2 e E3) que, por sua vez, também são circuitos.



Circuitos combinacionais:

Circuitos combinacionais são **combinações de portas/circuitos lógicos simples**. Dois ou mais circuitos se ligam a outro que, por sua vez, recebe como entradas apenas as saídas dos circuitos ligados a eles para fazer sua operação. Vejamos um exemplo abaixo do circuito $AND + AND = AND \ OR \ AND \rightarrow Y = AB + CD$.



Equações booleanas

Equações booleanas são funções que **especificam um circuito baseado em seus sinais**. Por exemplo:

$$\begin{array}{lcl}
 S = F(A, B, C_{in}) & \begin{array}{c} A \\ B \\ C_{in} \end{array} \begin{array}{c} \text{---} \\ \text{---} \\ \text{---} \end{array} \begin{array}{c} \text{---} \\ \text{---} \\ \text{---} \end{array} \begin{array}{c} S \\ C_{out} \end{array} & \begin{array}{c} S \\ C_{out} \end{array} = \begin{array}{c} A \oplus B \oplus C_{in} \\ AB + AC_{in} + BC_{in} \end{array}
 \end{array}$$

Obs.: AND, em equações, é representado como multiplicação. Exemplo: AB é A and B.

Obs. 2: A ordem de precedência das operações é (), NOT, AND e OR.

Definições:

- **Complemento (negação):** sinal com a “barrinha” em cima (NOT).
 - $\bar{A}, \bar{B}, \bar{C}$
- **Mintermo:** produto lógico (AND) que inclui todos os sinais de entrada aparecendo uma única vez, sejam eles complementos ou não. Além disso, os mintermos sempre representam uma linha da tabela verdade do circuito onde o resultado é 1 (true).
 - $ABC, A\bar{B}C, AB\bar{C}$
- **Implicante:** qualquer produto lógico (AND) de sinais, sejam eles complementos ou não. Implicantes são simplificações de mintermos e representam agrupamentos no mapa de Karnaugh.
 - $A\bar{B}C, \bar{A}C, BC$

Soma dos Produtos (SOP)

SOP é uma maneira de escrever equações booleanas, e toda equação pode ser escrita na forma SOP. Cada linha possui um **mintermo**. O mintermo é um produto (AND) de sinais. Cada mintermo é TRUE (1) para aquela linha (e apenas aquela linha). Construímos a função fazendo OR dos mintermos onde a saída é TRUE (1). Portanto, uma soma (OR) de produtos (AND).

A	B	Y	minterm	minterm name
0	0	0	$\bar{A} \bar{B}$	m_0
0	1	1	$\bar{A} B$	m_1
1	0	0	$A \bar{B}$	m_2
1	1	1	$A B$	m_3

$$Y = F(A, B) = \bar{A}\bar{B} + AB = \Sigma(1, 3)$$

Vejamos um exemplo simples para representar melhor uma situação lógica para a montagem de uma tabela verdade e o uso de mintermos no modelo SOP.

Suponhamos que fomos à padaria almoçar. Nós **não** iremos almoçar ($\bar{E}$) se **não** estiver aberto ($\bar{O}$) ou se só tiver coxinha (C). A tabela verdade que decide se iremos almoçar e a equação booleana com mintermos que representa isso é a seguinte:

O	C	E	minterm
0	0	0	$\bar{O} \bar{C}$
0	1	0	$\bar{O} C$
1	0	1	$O \bar{C}$
1	1	0	$O C$

$$E = O\bar{C} = \Sigma(2)$$

Álgebra booleana

A álgebra booleana usa axiomas e teoremas para **simplificar** equações booleanas. Ela funciona como a álgebra regular, mas mais simples: variáveis só tem dois valores (1 ou 0). **Dualidade** nos axiomas e teoremas: ANDs e ORs, 0's e 1's podem ser invertidos em conjunto sem manter o resultado lógico do circuito.

Axiomas/postulados booleanos:

Os axiomas da álgebra booleana são os princípios básicos que precisamos levar em consideração para fazermos todas as operações com as equações que serão originadas de tabelas e circuitos. Em geral, os axiomas são basicamente a lógica booleana traduzida em forma de equações.

	Axiom		Dual	Name
A1	$B = 0 \text{ if } B \neq 1$	A1'	$B = 1 \text{ if } B \neq 0$	Binary field
A2	$\overline{0} = 1$	A2'	$\overline{1} = 0$	NOT
A3	$0 \bullet 0 = 0$	A3'	$1 + 1 = 1$	AND/OR
A4	$1 \bullet 1 = 1$	A4'	$0 + 0 = 0$	AND/OR
A5	$0 \bullet 1 = 1 \bullet 0 = 0$	A5'	$1 + 0 = 0 + 1 = 1$	AND/OR

Teoremas booleanos:

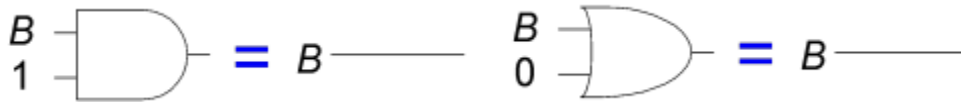
Os teoremas, por outro lado, são regras fundamentais da álgebra booleana usados para analisar, simplificar e otimizar expressões lógicas ou circuitos digitais. Eles se subdividem nos teoremas de uma (T1 a T5) e múltiplas variáveis (T6 a T12). A seguir, iremos passar por cada um deles e demonstrar seu uso com variáveis e circuitos.

- Teoremas de 1 variável:

	Theorem		Dual	Name
T1	$B \bullet 1 = B$	T1'	$B + 0 = B$	Identity
T2	$B \bullet 0 = 0$	T2'	$B + 1 = 1$	Null Element
T3	$B \bullet B = B$	T3'	$B + B = B$	Idempotency
T4		$\overline{\overline{B}} = B$		Involution
T5	$B \bullet \overline{B} = 0$	T5'	$B + \overline{B} = 1$	Complements

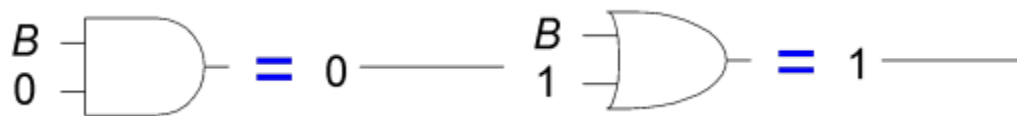
- **Teorema da Identidade:**

$B * 1 = B$, assim como $B + 0 = B$. Isso se dá porque, se $B = 0$, $B * 1 = 0$, já que $0 \text{ AND } 1 = 0$, assim como $B * 0 = 0$. Além disso, $B + 0 = 0$, pois $0 \text{ OR } 0$ também resulta em 0. De forma análoga, se $B = 1$, $B * 1 = 1$, dado que $1 \text{ AND } 1 = 1$, e $B + 0 = 1$, levando em consideração que $0 \text{ OR } 1 = 1$.



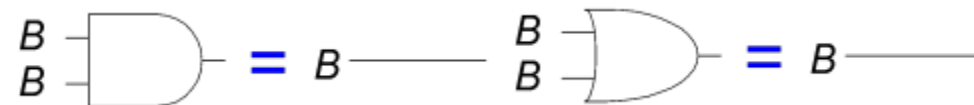
- **Teorema do Elemento Nulo:**

$B * 0 = 0$, assim como $B + 1 = 1$. Isso se dá porque um AND só possui resultado verdadeiro se ambos os sinais de entradas são 1 e, em um OR, pelo menos um 1 resulta em 1. Portanto, se $B = 1$, $B + 1 = 1$, assim como se $B = 0$, pois tanto $0 \text{ OR } 1$ quanto $1 \text{ OR } 1$ culminam em 1. Igualmente, $B * 0$ é sempre 0 pois $1 \text{ AND } 0$ e $0 \text{ AND } 0$ sempre culminam em 0.



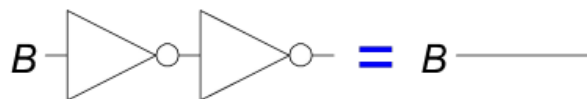
- **Teorema da Idempotência:**

$B * B = B$, assim como $B + B = B$. Isso se dá porque sinais iguais sempre culminam neles mesmos, como podemos ver com $0 \text{ AND } 0 = 0$, $1 \text{ AND } 1 = 1$, $0 \text{ OR } 0 = 0$ e $1 \text{ OR } 1 = 1$.



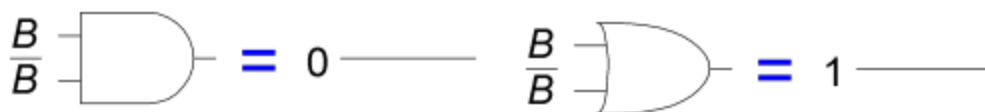
- **Teorema da Identidade/Involução:**

$\bar{\bar{B}} (\sim\sim B \text{ ou } \sim\bar{B}) = B$. Isso se dá porque uma dupla negação faz com que o sinal de entrada volte ao sinal original, já que o inverte duas vezes e $\sim 0 \rightarrow 1$ e $\sim 1 \rightarrow 0$, portanto, $\sim\sim 0 \rightarrow 0$.



- **Teorema do Complemento:**

$B * \bar{B} = 0$, assim como $B + \bar{B} = 1$. Isso se dá porque, no AND, sinais opostos sempre resultam em 0 e, no OR, sempre resultam em verdadeiro, como podemos ver se analisarmos $0 \text{ AND } 1 = 0$ e $1 \text{ OR } 0 = 1$ (respeitando a comutatividade).



- Teoremas de múltiplas variáveis:

	Theorem		Dual	Name
T6	$B \bullet C = C \bullet B$	T6'	$B + C = C + B$	Commutativity
T7	$(B \bullet C) \bullet D = B \bullet (C \bullet D)$	T7'	$(B + C) + D = B + (C + D)$	Associativity
T8	$(B \bullet C) + (B \bullet D) = B \bullet (C + D)$	T8'	$(B + C) \bullet (B + D) = B + (C \bullet D)$	Distributivity
T9	$B \bullet (B + C) = B$	T9'	$B + (B \bullet C) = B$	Covering
T10	$(B \bullet C) + (B \bullet \overline{C}) = B$	T10'	$(B + C) \bullet (B + \overline{C}) = B$	Combining
T11	$(B \bullet C) + (\overline{B} \bullet D) + (C \bullet D)$ $= B \bullet C + \overline{B} \bullet D$	T11'	$(B + C) \bullet (\overline{B} + D) \bullet (C + D)$ $= (B + C) \bullet (\overline{B} + D)$	Consensus
T12	$\overline{B_0 \bullet B_1 \bullet B_2 \dots}$ $= (\overline{B_0} + \overline{B_1} + \overline{B_2} \dots)$	T12'	$\overline{B_0 + B_1 + B_2 \dots}$ $= (\overline{B_0} \bullet \overline{B_1} \bullet \overline{B_2} \dots)$	De Morgan's Theorem

- Teorema da Comutatividade:

$B * C = C * B$, assim como $B + C = C + B$. Isso se dá porque não importa a ordem de entrada dos sinais, a saída é sempre a mesma.

- Teorema da Associatividade:

$(B * C) * D = B * (C * D)$, assim como $(B + C) + D = B + (C + D)$. Isso porque, em operações da mesma natureza, a ordem de precedência não importa.

- Teorema da Distribuição:

$(B * C) + (B * D) = B * (C + D)$, assim como $(B + C) * (B + D) = B + (C * D)$.

Logo de cara, é importante notar que T8' é uma propriedade exclusiva da álgebra booleana que não se aplica à álgebra convencional, visto que esta não engloba a distributiva de adições. No entanto, isso se dá porque, em álgebra booleana, podemos colocar variáveis em evidência independentemente da operação associada a ela (contanto que os blocos conttenham operações internas opostas às que os ligam com os outros blocos). Para fim de prova, podemos adotar $B = 0$ e observar que, quando fazemos um OR de 2 ANDs envolvendo 0, isso é equivalente a fazer um AND entre 0 e o OR dos outros sinais, admitindo que isso os anulará, nesse caso, devido à propriedade do axioma A2. Da mesma forma, se fizermos um AND com dois ORs, isso é equivalente a ter um OR com um AND da variação dos outros sinais, visto que um 0 em um OR só resultará em 0 se a outra entrada também for 0.

- **Teorema da Cobertura:**

$B * (B + C) = B$, assim como $B + (B * C) = B$. Isso se dá porque as expressões apresentam uma união das duas principais operações booleanas que podem determinar o resultado final da equação. Por exemplo, se tomarmos B como 0, em T9, o resultado final será 0 pois qualquer AND com 0 é 0. Com efeito, em T9', $B = 0$ fará com que o AND interno resulte em 0 e, no OR final, $0 + 0 = 0$. Para comprovar completamente, se tomarmos B como 1, em T9, o OR interno será obrigatoriamente 1, fazendo com que o AND também resulte em 1 por culminar em $1 * 1 = 1$. Com efeito, em T9', $B = 1$ fará com que o resultado final seja 1 pois qualquer OR com 1 é 1.

- **Teorema da Combinação:**

$(B * C) + (B * \bar{C}) = B$, assim como $(B + C) * (B + \bar{C}) = B$. Isso se dá porque, se analisarmos com profundidade as operações propostas e aplicarmos a ela o teorema da distribuição, temos que $B * (C + \bar{C})$ e $B + (C * \bar{C})$. No primeiro caso, $C + \bar{C}$ é sempre 1, o que fará com que o resultado da primeira expressão também seja 1, se $B = 1$, ou 0, se $B = 0$. No segundo, $C * \bar{C}$ se anulam e resultam em 0, fazendo com que o resultado da expressão seja 0 se $B = 0$, ou 1, se $B = 1$.

- **Teorema do Consenso:**

$(B * C) + (\bar{B} * D) + (C * D) = B * C + \bar{B} * D$, assim como $(B + C) * (\bar{B} + D) * (C + D) = (B + C) * (\bar{B} + D)$. Isso se dá porque, como, **no primeiro caso**, os três produtos estão conectados em ORs, basta resolvermos um deles para que a condição se satisfaça. Portanto, se adotarmos, por exemplo, $B = 1$, teremos $(1 * C) + (0 * D) + (C * D)$, o que fará com que os resultados fiquem agora à mercê de C e D. No entanto, se tomarmos $C = 1$ e $D = 1$, o que garantiria a saída 1 de CD, podemos notar que isso faria com que BC também tivesse uma saída 1, o que já daria a resposta positiva para o circuito sem que CD precisasse sequer ser considerado. O mesmo se aplica se $B = 0$ e, novamente, $C = 1$ e $D = 1$ (a fim de provar um caso de relevância do implicante), pois teríamos $(0 * 1) + (1 * 1) + (1 * 1)$. Dessa vez, o conjunto $\bar{B} + D$ já garante a saída 1 para o circuito e, novamente, CD nem precisou ser levado em consideração. Já para o **caso contrário**, $(C + D)$ é descartado pois, se tivermos $B = 1$, temos $(1 + C) * (0 + D) * (C + D)$, já observamos que a saída do primeiro conjunto será 1, mas o restante ainda está à mercê de C e D. Se pensarmos nas hipóteses de descarte do circuito (saída 0), basta tomar $C = 0$ ou $D = 0$ que o circuito será descartado antes mesmo de chegar no último conjunto, ressaltando sua irrelevância.

Se quisermos **comprovar** esse teorema de outra maneira, podemos simplesmente **simplificá-lo** até obtermos seu resultado. Para isso, temos $BC + \bar{B}D + CD$. Se multiplicarmos CD por $(B + \bar{B})$ (que resulta em 1 e não afeta o valor do mintermo), teremos mais sinais para fazermos a distributiva, uma vez que chegaremos em $BC + \bar{B}D + BCD + \bar{B}CD$. A partir daí, podemos agrupar os sinais por BC e $\bar{B}D$,

fazendo com que a expressão fique $BC * (1 + D) + \overline{B}D * (1 + C)$. Como todo OR com 1 é 1, os parênteses se anulam e temos $BC + \overline{B}D$.

- **Teorema de De Morgan:**

$$\overline{B_0 \bullet B_1 \bullet B_2 \dots} = (\overline{B_0} + \overline{B_1} + \overline{B_2} \dots), \text{ assim como } \overline{B_0 + B_1 + B_2 \dots} = (\overline{B_0} \bullet \overline{B_1} \bullet \overline{B_2} \dots). \text{ Traduzindo}$$

melhor, isso significa que o complemento de um **conjunto de ANDs** com **n entradas verdadeiras** culmina em um **OR** com **n entradas complementares**, da mesma forma que o complemento de um **conjunto de ORs** com **n entradas verdadeiras** culmina em um **AND** com **n entradas complementares**. Pensando em um exemplo mais abstrato para comprovarmos o teorema, podemos pensar em pré condições. Suponhamos que, para conseguir um emprego, eu preciso passar na entrevista e na prova teórica. Isso significa que, se eu **negar** o conjunto, ou seja, para eu **NÃO** passar na entrevista, **OU** eu **NÃO** passei na entrevista, **OU** eu **NÃO** passei na prova teórica. Pensando agora no modelo contrário, se para conseguir o emprego eu precisasse passar na prova teórica ou na prova prática, para eu **NÃO** conseguir o emprego, isso significa que eu **NÃO** passei na prova teórica e **NÃO** passei na prova prática.

- **Teorema da Absorção:**

$A + \overline{A}B = A + B$, assim como $A * (\overline{A} + B) = AB$. Isso se dá porque, se nos apegarmos aos princípios do teorema da distribuição e da associatividade, podemos interpretar $A + \overline{A}B$ como sendo $A + [\overline{A}(B)]$, que pode então ser distribuído para $(\overline{A} + A) * (A + B)$. Como $\overline{A} + A = 1$, o resultado da simplificação é $A + B$. Vale ressaltar que essa propriedade também pode ser aplicada se $\overline{A}$ estiver associado a mais entradas, já que isso culminará apenas no resultado final ser $A + \text{*outros sinais de entrada*}$.

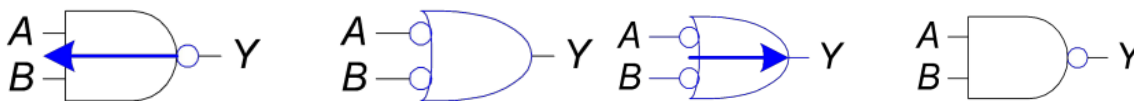
Simplificando equações:

Para colocarmos em prática os conceitos vistos dos teoremas, vamos aplicá-los nas equações a seguir a fim de encontrar a equação mais simples possível e, consequentemente, o menor circuito necessário.

- **$Y = AB + \overline{A}B$:**
 - $Y = B * (A + \overline{A}) \rightarrow T8$
 - $Y = B * (1) \rightarrow T5$
 - $Y = B \rightarrow T1$
- **$Y = A(AB + ABC)$:**
 - $Y = A * A (B + BC) \rightarrow T8$
 - $Y = A * (B + BC) \rightarrow T3$
 - $Y = A * B * (1 + C) \rightarrow T8$
 - $Y = A * B * (1) \rightarrow T2$
 - $Y = AB \rightarrow T1$

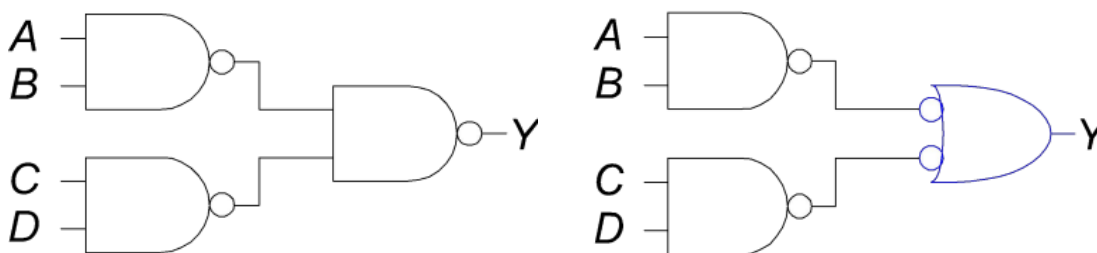
Empurrando a bolha/Bubble Pushing:

Uma forma de modificarmos circuitos para facilitar sua compreensão ou melhorar sua construção é modificando a posição dos inversores de sinal, ou seja, mudando a posição da “bolinha” que representa o NOT e colocando-a nas entradas ou na saída. No entanto, essa mudança de posição **muda o tipo de operador**, ou seja, para mudarmos as bolinhas de posição, um AND se torna OR e assim por diante.



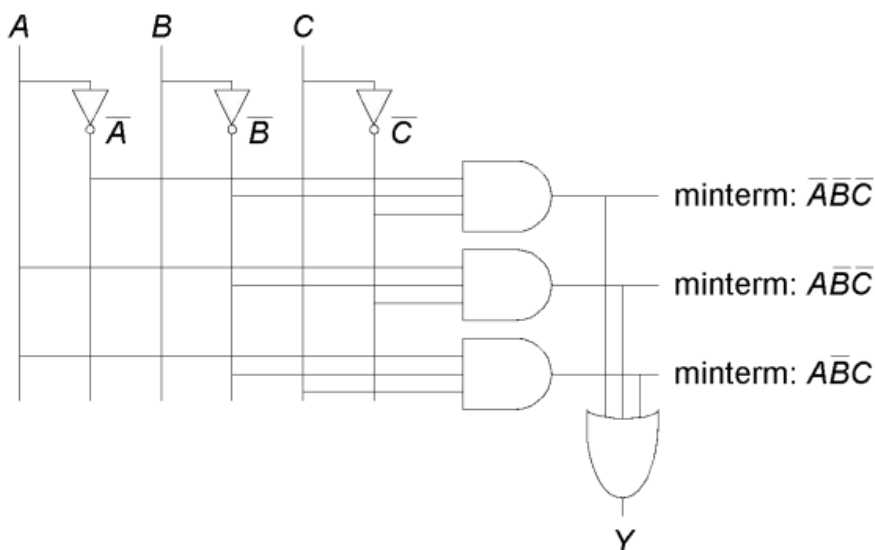
- **Aplicando em circuitos maiores:**

Vamos considerar o seguinte circuito com 3 NANDs. Se analisarmos bem e aplicarmos os princípios do bubble pushing, podemos usar 2 NANDs e 1 OR com entradas negadas. Isso fará com que a negação das ANDs se anule com a negação da entrada da OR, culminando em 2 ANDs e 1 OR ($Y = AB + CD$).



Convertendo equações em circuitos

Apesar de já termos visto exemplos da conversão de equações em circuitos lógicos, vejamos a seguir a conversão da equação $Y = \bar{A}\bar{B}\bar{C} + \bar{A}B\bar{C} + A\bar{B}\bar{C}$.

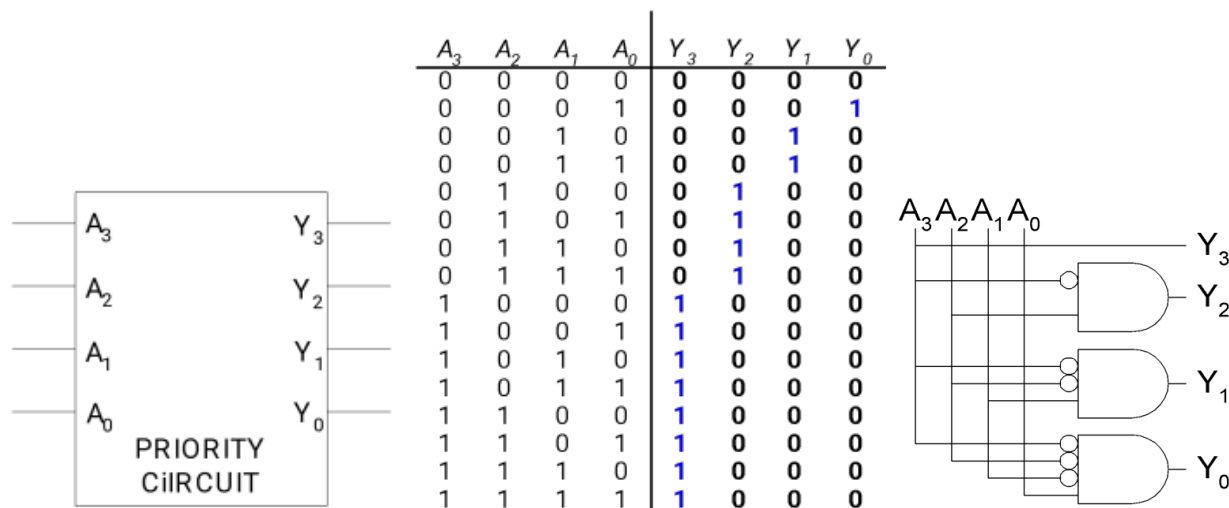


09 - Simplificação de Circuitos Lógicos

Circuitos de múltiplas saídas

Circuito de Prioridade:

Um circuito de prioridade é um sistema lógico combinacional que recebe várias entradas simultâneas e gera um código binário baseado apenas na entrada ativa (cujo sinal é 1) de maior posição na hierarquia. Isso significa que, se temos um circuito cuja prioridade segue a ordem $A_3 \rightarrow A_2 \rightarrow A_1 \rightarrow A_0$ e possui uma saída para cada entrada mas somente uma das saídas pode receber o valor positivo (saída determinada), se tivermos A_3 com sinal 1, Y_3 receberá 1 e Y_2 , Y_1 e Y_0 ficarão com 0, e assim por diante, como podemos ver abaixo:



Outra interpretação sobre esses circuitos pode ser obtida sob a lógica **Don't Care**, ou seja, quando o de maior precedência for verdadeiro, todos aqueles que são inferiores na hierarquia são descartados pelo circuito, como podemos ver na tabela abaixo (que é análoga à tabela verdade anterior):

A_3	A_2	A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	X	0	0	1	0
0	1	X	X	0	1	0	0
1	X	X	X	1	0	0	0

Mapa de Karnaugh

Mapas de Karnaugh são uma técnica visual de simplificação/minimização de equações booleanas de múltiplas entradas (sendo viável até 4) que consiste na montagem de um diagrama baseado na tabela verdade de um circuito específico. A ideia que sustenta essa metodologia é a de que se tivermos um conjunto de entradas que variam em operações cuja saída é 1 e um outro que se mantém, os responsáveis por determinar o circuito são justamente os que se mantêm, enquanto aqueles que variam são irrelevantes para o circuito. Traduzindo melhor, ele segue a lógica de que **$PA + P\bar{A} = P$** , visto que, se A muda de valor entre os dois mintermos, quem de fato determinará a saída do circuito é o sinal de P.

Explicando de forma mais detalhada, um mapa de Karnaugh (ou K-map) é uma matriz de **2^n quadrados**, sendo **n** o número de **variáveis de entrada (sinais)** circuito. Ele organiza as combinações da tabela-verdade de forma que quadrados fisicamente adjacentes diferem em apenas uma variável, ou seja, quando montamos o mapa com as combinações dos valores da tabela verdade, apenas um sinal dentre todos do conjunto pode se diferenciar do conjunto ao lado. Por exemplo: se tivermos sinais binários de 00 a 11, ao invés de colocarmos na ordem binária convencional (00, 01, 10 e 11), usamos o **gray code** para que a sequência não ultrapasse as regras do mapa, fazendo com que a sequência seja 00, 01, 11 e 10 (uma forma mais lúdica de pensarmos na montagem do mapa é que os elementos devem ser “vizinhos”, ou seja, o primeiro elemento do quadrado ao lado deve ser igual ao segundo do quadrado atual).

Além disso, o objetivo de um K-map é agrupar os valores 1 (mintermos) para eliminar variáveis que mudam de estado (de 0 para 1) dentro do grupo, usando a premissa do Teorema da Combinação ($PA + P\bar{A} = P$) para realizar sua operação.

Regras de montagem:

Para construir um mapa de Karnaugh e usufruir de sua funcionalidade, além do uso de gray code para ordenar os campos da matriz, existem regras para que esse método funcione corretamente:

1. Cada 1 no mapa deve ser circulado **pelo menos uma vez**;
2. Só podemos circular regiões **retangulares ou quadradas** em **potências de 2** (grupos de 1, 2, 4, 8, etc.);
3. Cada círculo deve ser **o maior possível**. Um grupo maior elimina mais variáveis, resultando em menos portas lógicas no circuito final;
4. Os círculos **podem ultrapassar as bordas** do mapa (como se este fosse um cilindro ou esfera), pois a borda direita é adjacente à esquerda e o topo é adjacente à base.
5. Um “Don’t Care” só pode ser circulado se ajudar a otimizar a equação (já que eles podem ser considerados como 1 para os fins do mapa).

Vejamos um exemplo de mapa de K-map a seguir:

A	B	C	Y
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

Y	C	AB			
		00	01	11	10
0	0	1	0	0	0
	1	1	0	0	0

Y	C	AB			
		00	01	11	10
0	0	$\bar{A}\bar{B}\bar{C}$	$\bar{A}B\bar{C}$	$AB\bar{C}$	$A\bar{B}\bar{C}$
	1	$\bar{A}\bar{B}C$	$\bar{A}BC$	ABC	$A\bar{B}C$

A	B	C	Y
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

Y	C	AB			
		00	01	11	10
0	0	1	0	0	0
	1	1	0	0	0

$$Y = \bar{A}\bar{B}$$

Mapa com 3 entradas:

Quando montamos mapas com 3 entradas, considerando a natureza da montagem do mapa como uma matriz de 2^n entradas, teremos 8 quadrados em uma matriz 2x4 em que agruparemos as 2 primeiras entradas e seus valores combinados na parte de cima e a terceira e seus valores ao lado, como mostra o exemplo abaixo:

Y	C	AB			
		00	01	11	10
0	0	$\bar{A}\bar{B}\bar{C}$	$\bar{A}B\bar{C}$	$AB\bar{C}$	$A\bar{B}\bar{C}$
	1	$\bar{A}\bar{B}C$	$\bar{A}BC$	ABC	$A\bar{B}C$

Se considerarmos a tabela verdade dada a seguir, o resultado do mapa que obteríamos seria o seguinte:

Truth Table			
A	B	C	Y
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

K-Map					
Y	C	AB			
		00	01	11	10
0	0	0	1	0	0
	1	0	1	1	0

$$Y = \bar{A}B + BC$$

Mapas com 4 entradas:

Mapas com 4 entradas se comportam seguindo exatamente os mesmos princípios anteriores, mas agora tendo 16 quadrados e usando agrupamentos em ambos os eixos. No exemplo a seguir, temos uma tabela verdade de 4 entradas e o mapa formado a partir dela:

A	B	C	D	Y
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

Y \ CD \ AB	00	01	11	10
00	1	0	0	1
01	0	1	0	1
11	1	1	0	0
10	1	1	0	1

Y \ CD \ AB	00	01	11	10
00	1	0	0	1
01	0	1	0	1
11	1	1	0	0
10	1	1	0	1

$Y = \bar{A}C + \bar{A}BD + \bar{A}\bar{B}C + \bar{B}\bar{D}$

Mapa com Don't Cares:

Quando, em um circuito, temos situações em que **a combinação específica** de entradas é **impossível de ocorrer** na vida real ou que a saída para aquela combinação simplesmente **não afeta o resto do sistema**, fazemos uso de **Don't Cares** (representados por X, D ou ?) tanto na tabela verdade quanto no mapa que será formado a partir dela. No entanto, como descrito nas regras de montagem dos mapas, só levamos em consideração o X se ele ajudar a tornar um grupo de 1s ainda maior (sendo considerado como um 1 também). Caso contrário, consideramos-o como 0 e assumimos sua irrelevância.

Vejamos um exemplo de uso a seguir:

A	B	C	D	Y
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	X
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	X
1	0	1	1	X
1	1	0	0	X
1	1	0	1	X
1	1	1	0	X
1	1	1	1	X

Y \ CD \ AB	00	01	11	10
00	1	0	X	1
01	0	X	X	1
11	1	1	X	X
10	1	1	X	X

Y \ CD \ AB	00	01	11	10
00	1	0	X	1
01	0	X	X	1
11	1	1	X	X
10	1	1	X	X

$Y = A + \bar{B}\bar{D} + C$

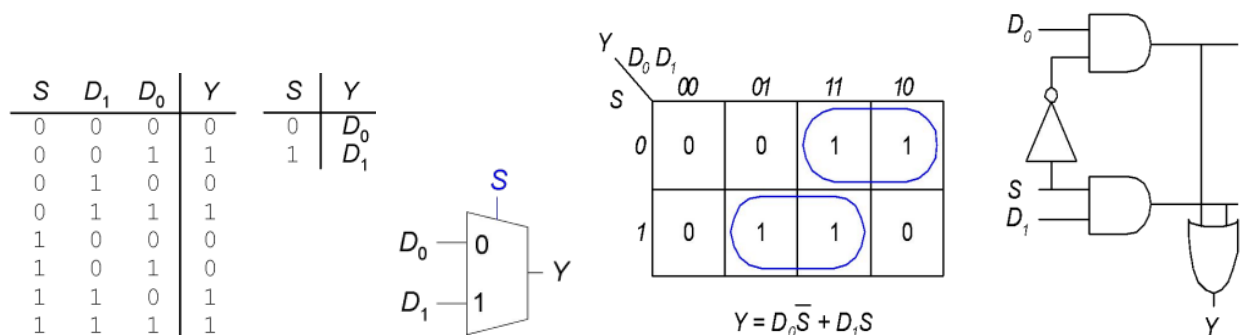
Circuitos Combinacionais Fundamentais

Multiplexadores:

Multiplexadores são um dos blocos combinacionais mais utilizados, sendo ele um **seletor de sinais** que, a partir de entradas possíveis, determina qual será escolhida a partir de sua combinação com o sinal seletor/de controle, ou seja, escolhe um de N entradas para se conectar à saída. Explicando melhor, o sinal de controle servirá para definir qual sinal de entrada deverá atuar sobre a escolha da saída.

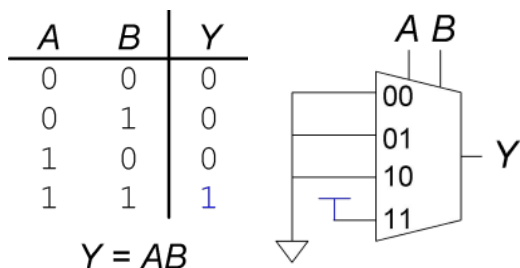
Para escolher entre **N entradas**, o circuito precisa de **$\log_2 N$ bits de seleção** (sinal S). Por exemplo, um Mux 2:1 possui 2 entradas de dados (D_0 e D_1) e um bit de seleção (S). Sua equação, por sua vez, é $Y = D_0\bar{S} + D_1S$, e seu funcionamento se dá da seguinte forma: se $S = 0$, a saída é D_0 ; se $S = 1$, a saída é D_1 .

Vejamos como é construído esse circuito a seguir:



- **Multiplexadores como Lookup Tables:**

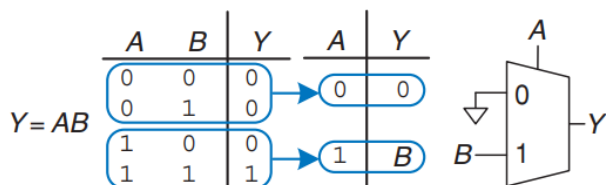
Multiplexadores podem ser usados como tabelas de consulta para implementar qualquer função lógica. Para isso, ligamos as variáveis de entrada nos pinos de seleção e conectamos as entradas de dados (D) a 0 ou 1, espelhando o comportamento da tabela verdade desejada. Vejamos o funcionamento disso no exemplo da função $Y = AB$:



- **Reduzindo o tamanho de multiplexadores:**

Assim como podemos converter esse tipo de circuito para lookup tables, podemos também simplificá-los para que consumam menos recursos. Para ilustrar melhor, vamos considerar um mux que representa a porta lógica AND de 2 entradas.

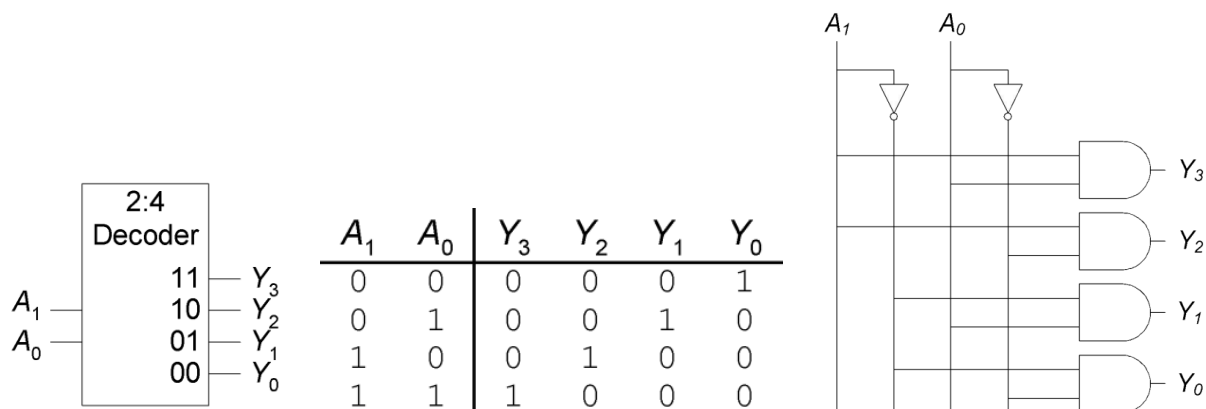
Para ela, quando $A = 0$, $Y = 0$ independentemente de B . Quando $A = 1$, $Y = 0$ se $B = 0$ e $Y = 1$ se $B = 1$, portanto, $Y = B$. A construção do mux fica, então, da seguinte forma:



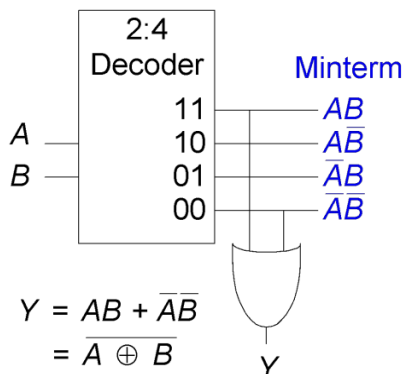
Decodificadores:

Enquanto multiplexadores são seletores convergentes (muitas entradas para uma saída), decodificadores são **expansores divergentes** (poucas entradas para muitas saídas), recebendo **N entradas** e possuindo **2^N linhas de saída**.

A regra principal do decodificador é que apenas **uma** única saída é ativada por vez, dependendo do padrão binário colocado na entrada. Essa regra é chamada de **one-hot** e serve para indicar que a única saída cujo resultado é diferente de 0 representa a saída ativa.



Cada uma das saídas de um decodificador representa exatamente um único mintermo da tabela-verdade. Por isso, se pegarmos um decodificador e conectarmos suas saídas específicas a uma porta lógica OR, conseguimos construir fisicamente a expressão de qualquer tabela-verdade.



10 - Circuitos aritméticos - Base da ALU

Estudar a Unidade Lógica Aritmética (ALU) é compreender o “coração” de processamento de um computador. A construção dos circuitos complexos que a constroem baseia-se em um princípio estrito de **hierarquia digital**: blocos menores e mais simples (portas lógicas) são combinados para formar submódulos (como somadores de 1 bit), que por sua vez formam circuitos maiores (como somadores multibit) até compor a ALU completa.

A seguir, veremos cada um dos principais circuitos aritméticos que se interligam na arquitetura de uma ALU.

Somador de 1 Bit (bloco fundamental)

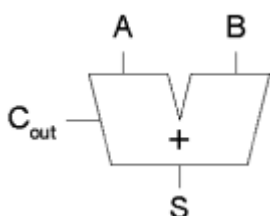
Antes de somar números grandes, o computador precisa saber somar a menor unidade de informação: o bit. Isso é feito por meio de duas variantes de circuitos:

Meio-Somador (Half Adder):

O Meio-Somador é o circuito aritmético mais simples. Ele recebe duas entradas de 1 Bit (A e B) e produz duas saídas: a Soma (S) e o Transporte de Saída/Carry-out/ C_{out} . (o famoso “vai 1”, que é passado quando pelo menos duas entradas são verdadeiras). No entanto, veremos que ele é limitado em relação ao somador completo, uma vez que ele **não possui uma entrada para o transporte vindo de uma coluna anterior** (também conhecido como Carry-in/ C_{in}). Portanto, ele só pode ser usado para somar o bit menos significativo (o primeiro bit à direita) de um número binário.

As equações booleanas que representam cada uma das saídas desse circuito são, respectivamente:

- $S = A \oplus B$ (Uma porta XOR, já que a saída é apenas 1 se as portas forem diferentes);
- $C_{out} = A * B$ (Uma porta AND, já que só haverá transporte se ambas forem 1).



A	B	C_{out}	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

$$S = A \oplus B$$
$$C_{out} = AB$$

Somador Completo (Full Adder):

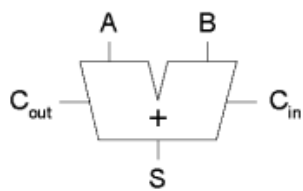
Para resolver a limitação do meio-somador, o full adder incorpora uma **terceira entrada: o Carry-in** (que pode ser entendido como o “vem 1”). Isso permite que ele seja encadeado com outros somadores para processar números de múltiplos bits.

As equações booleanas que representam cada uma das saídas desse circuito são, respectivamente:

- $S = A \oplus B \oplus C_{in}$ (A soma bit a bit é a operação XOR entre as três variáveis);
- $C_{out} = AB + AC_{in} + BC_{in}$ (Gera transporte se pelo menos duas das três entradas forem iguais a 1).

Internamente, um Full Adder pode ser construído interligando 2 Half Adders e uma porta OR.

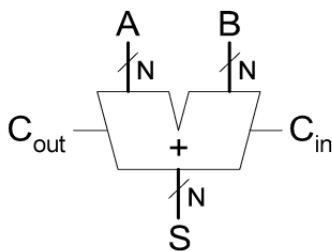
C_{in}	A	B	C_{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1


$$S = A \oplus B \oplus C_{in}$$
$$C_{out} = AB + AC_{in} + BC_{in}$$

Somadores Multibit (Carry Propagate Adders - CPAs)

Para realizar a operação aritmética de adição em números de tamanhos reais de palavra de um processador (como 8, 16, 32 ou 64), múltiplos somadores completos de 1 bit precisam trabalhar em conjunto. A forma como o transporte (carry) transita entre esses blocos define o desempenho e o custo do hardware.

O símbolo que os representa é o seguinte (N representa a quantidade de bits):

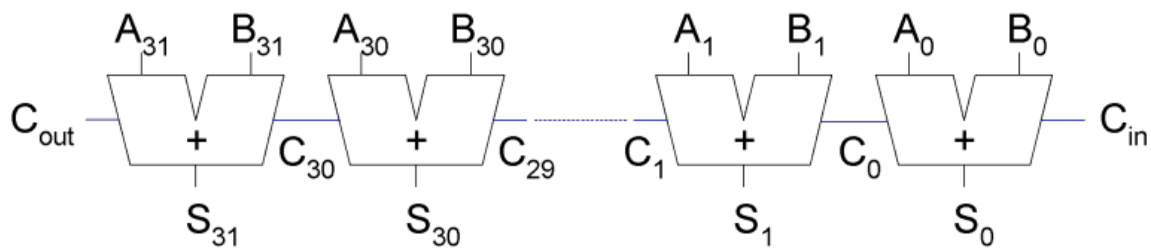


Somador de Propagação em Cadeia (Ripple-Carry Adder):

Nesta estrutura elementar, **o C_{out} de um somador de 1 bit é conectado diretamente ao C_{in} do somador do próximo bit à esquerda**. Seu funcionamento é muito simples e idêntico a uma soma feita no papel: somamos a primeira coluna (bits A

e B), o “vai um” sobe para a segunda/próxima coluna, realizamos a soma da segunda junto com o “vai um” e assim por diante.

A maior vantagem associada a esse tipo de construção é que o **hardware é extremamente simples**, linear e intuitivo de projetar. No entanto, é um sistema muito **lento**, uma vez que o bit mais significativo (por exemplo, o bit 31 em uma palavra de 32 bits) não pode calcular seu resultado final até que o carry tenha sido propagado (“ondulado” ou rippled) por todos os 31 somadores anteriores. Isso cria um longo atraso de propagação (propagation delay), limitando a frequência máxima de clock do processador.



Somador de Antecipação de Transporte (Carry-Lookahead Adder - CLA):

Para eliminar a dependência sequencial do circuito Ripple-Carry, o CLA calcula os transportes em paralelo utilizando lógica booleana complexa baseada em dois conceitos:

- **Geração (G_i):** O bit i vai gerar um transporte de saída independentemente do transporte que vem de trás se A_i e B_i forem 1 ($G_i = A_i * B_i$);
- **Propagação (P_i):** O bit i vai propagar o transporte que vem de trás se pelo menos uma das entradas A_i ou B_i for 1 ($P_i = A_i + B_i$).

A partir disso, as equações calculam o transporte de qualquer estágio de forma direta. Por exemplo: $C_1 = G_0 + P_0 C_0$, assim como $C_2 = G_1 + P_1 C_1 = G_1 + P_1(G_0 + P_0 C_0) = G_1 + P_1 G_0 + P_1 P_0 C_0$.

Generalizando, o carry out da coluna i é $C_i = A_i B_i + (A_i + B_i) C_{(i-1)} = G_i + P_i C_{(i-1)}$.

Este circuito é muito mais rápido que o Ripple Carry porque o tempo de cálculo passa a ser praticamente constante, mas exige uma quantidade massiva de portas lógicas adicionais conforme o tamanho da palavra cresce.

Exemplo: blocos de 4 bits ($G_{3:0}$ and $P_{3:0}$): **Generalizando,**

$$G_{3:0} = G_3 + P_3 (G_2 + P_2 (G_1 + P_1 G_0))$$

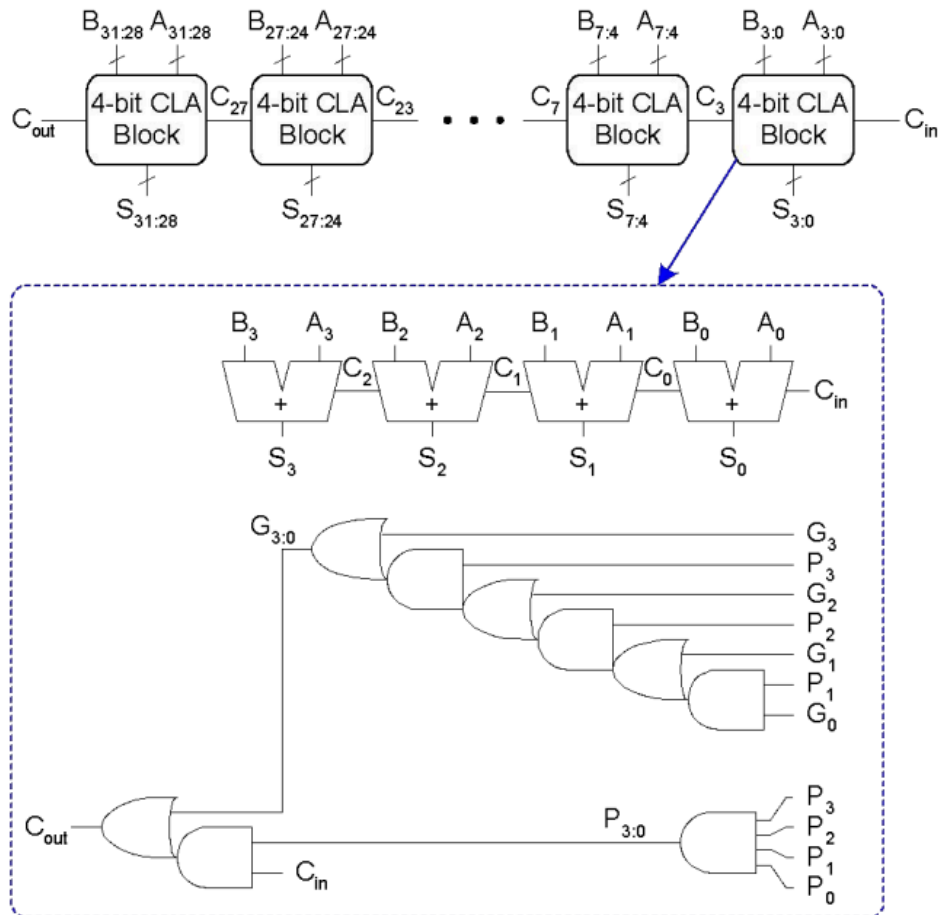
$$P_{3:0} = P_3 P_2 P_1 P_0$$

$$G_{ij} = G_i + P_i (G_{i-1} + P_{i-1} (G_{i-2} + P_{i-2} G_j))$$

$$P_{ij} = P_i P_{i-1} P_{i-2} P_j$$

$$C_i = G_{ij} + P_{ij} C_{j-1}$$

O circuito que descreve um CLA pode ser visto na imagem a seguir:



Somador de Prefixo (Prefix Adder):

Um somador de prefixo representa o estado da arte em somadores multibit para palavras grandes (32 ou 64 bits). Ele realiza o cálculo de geração e propagação usando uma estrutura em árvore lógica de prefixos binários (como as árvores de Kogge-Stone ou Brent-Kung).

Esse tipo de somador oferece o **desempenho máximo** (velocidade logarítmica), sendo ideal para caminhos de dados de alta velocidade, porém consome uma área física substancial no chip de silício devido à densidade de fiação e hardware.

O Design e a Interconexão de uma ALU Simples

Uma ALU não faz apenas adições; ela executa múltiplas operações lógicas e aritméticas compartilhando a mesma estrutura de hardware fundamental. Para fazer isso, o circuito combina portas lógicas (AND, OR), inversores, um somador multibit e multiplexadores de forma estratégica. Para saber qual operação ela deve realizar, a ALU faz uso de 3 bits de controle de função, cuja notação é $F_{2:0}$. Abaixo está o mapeamento de como ela responde a eles:

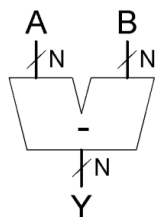
O Mecanismo de Subtração (A - B):

Na matemática binária por Complemento a Dois, subtrair um número é o mesmo que somar o inverso dele adicionado de 1: $A - B = A + (-B) = A + \bar{B} + 1$. Para efetuar isso na ALU com o controle $F_{2,0} = 110$, o hardware executa duas ações automáticas em paralelo baseadas no bit mais significativo do controle ($F_2 = 1$):

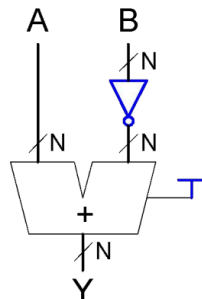
1. **Inversão de B:** A entrada B passa por um multiplexador de entrada controlado por F_2 . Como $F_2 = 1$, o Mux seleciona a entrada invertida ($\bar{B}$, ou seja, aplica-se a operação NOT em todos os bits de B).
2. **Injeção do “+1”:** A linha do bit de controle F_2 está conectada diretamente à entrada de transporte inicial (C_{in} do primeiro bit) do somador multibit. Quando $F_2 = 1$, esse “1” entra na conta, somando-se ao primeiro estágio.

Dessa forma, o somador executa fisicamente a operação $A + \bar{B} + 1$, entregando na saída do circuito o valor exato de $A - B$ de forma transparente.

Symbol



Implementation

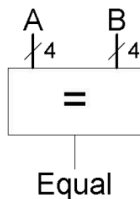


Circuitos Comparadores:

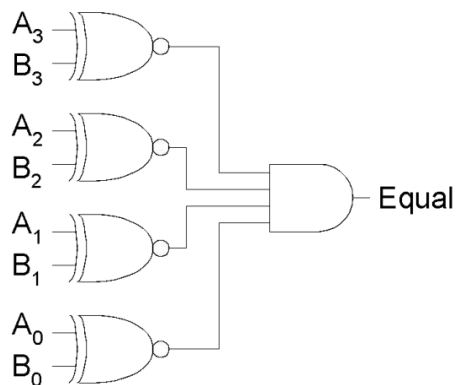
Outro tipo de circuitos muito importantes não só para a ALU mas para várias outras operações são os comparadores. Os dois principais são os comparadores de igualdade e o “menor que”, que é usado na ALU para a operação SLT, que veremos mais para frente.

O **comparador de igualdade** funciona através da **união de XNORs em um AND**, uma vez que os XNORs só retornarão 1 se os caracteres forem iguais e o AND garante que não haja nenhuma divergência. Vejamos sua implementação quando temos 4 bits de entrada:

Symbol

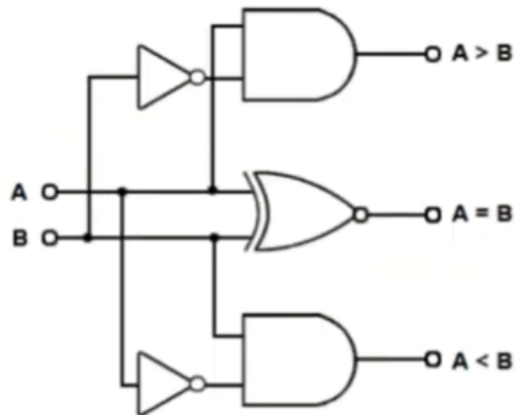


Implementation

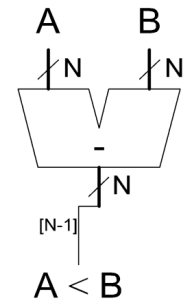


O **comparador “menor que”**, por sua vez, funciona comparando o inverso do bit A ($\bar{A}$) com o bit B em uma porta AND, isso porque, se $A = 1$ e $B = 0$ ($A < B$), teremos que o inverso do A é 0, satisfazendo a condição da porta AND e resultando em saída 1 para $A < B$.

Esse comparador é parte do **comparador de magnitude**, que realiza tanto a comparação de igualdade quanto as de “maior e menor que”, conforme podemos ver na imagem a seguir:



No caso de $A > B$ (o “**maior que**”), a lógica é praticamente idêntica a $A < B$, já que agora invertemos o sinal de B ($\bar{B}$), uma vez que, se $A = 0$ e $B = 1$ ($A < B$), o inverso de B será 0 também, satisfazendo a condição da porta AND e resultando em saída 1 para $A > B$.



O Mecanismo da Operação Set Less Than (SLT):

A operação SLT serve para avaliar condições (ex: if ($A < B$)). O objetivo dela é retornar o valor booleano verdadeiro (1 em formato de 32 bits, isto é, 0x00000001, que é convertido através da porta lógica **zero extend**, que adiciona 0s à esquerda do número) se A for menor que B, ou false (0x00000000) caso contrário. Para que isso seja realizado, o controle deve ser configurado como $F_{2:0} = 111$ para indicar a operação. Vale ressaltar que, apesar de existir, esse modelo de ALU não faz uso de comparadores de magnitude para obter o resultado de SLT, uma vez que se baseia na subtração de A e B e no uso do sinal (bit mais significativo) do resultado para definir a saída final (se for 1, é menor que).

Vamos ver o passo a passo do funcionamento desse mecanismo usando como base os valores $A = 25$ e $B = 32$:

1. **Subtração Interna:** Como o bit de controle F_2 é igual a 1, a ALU inverte B, define o C_{in} como 1 e força o bloco a calcular uma subtração ($A - B$). No exemplo, calcula-se $25 - 32 = -7$.
2. **O Bit de Sinal:** Em representação de Complemento a Dois para números com sinal, o bit mais significativo de uma palavra (o bit S_{31} em arquiteturas de 32 bits) é o bit de sinal.
 - a. Se o resultado for negativo (o que significa matematicamente que $A < B$), o bit S_{31} será obrigatoriamente 1;

- b. Se o resultado for zero ou positivo ($A \geq B$), o Bit S_{31} será igual a 0.
- 3. A Interconexão de Redirecionamento:** No circuito da ALU de múltiplos bits, a saída do multiplexador final para a função 111 (SLT) possui uma fiação muito específica:
- Para o bit 0 (o menos significativo da saída geral), a entrada do mux correspondente ao SLT recebe o fio vindo diretamente da saída de soma do bit S_{31} ;
 - Para todos os outros bits de 1 a 31, a entrada do mux correspondente ao SLT está permanentemente aterrada (conectada ao nível lógico 0).

Resultado do exemplo $25 < 32$: Como -7 é um número negativo, o somador do 31º bit gera um resultado onde $S_{31} = 1$. Pelas conexões internas da ALU, esse 1 é jogado diretamente na posição do bit 0 da saída final, enquanto as posições de 1 a 31 recebem 0. A saída consolidada da ALU será o número 0x00000001 (representação do valor lógico verdadeiro), concluindo a operação com máxima eficiência de hardware.

Datapaths na ALU:

A seguir, vamos conferir o datapath traçado na ALU para as operações OR, AND, adição, subtração e SLT.

